252-0027

Introduction to Programming

Exercises

Week 7: Hoare Logic, Problem Solving

Jan Leonard
Department of Computer Science
ETH Zurich

Plan for today

- Organization
- Review Ex. 1
- Preview
- Weakest Precondition with branching
- Hoare Triple
- Permutation Check Exercise
- Kahoot
- **NOT** Towers of Hanoi

Organization

- My name: Jan Leonard
- For questions: leonardj@ethz.ch
  - Please put «[EProg26]» in the subject
- New exercises: Tuesday evening (normally)
- Submit exercises by Tuesday evening (23:59) following week
  - Always submit via Git
  - Solutions in a separate Git project



Mailing List

docs.google.com



Eprog TA — HS 2026

janleonard115.github.io

Exam Format

- Theory Part only in Moodle
- Programming Part is gonna be printed in German
  - With some copies in English on request
  - Enough copies are gonna be at the exam (don’t worry)
- Programming Part in German and English as a pdf

Timed Bonus

- For this week the bonus exercise will be published next Tuesday 3rd of November at 5pm
- You will have two hours to submit
- This simulates the exam
- Submit often!
- Take this seriously - it is a nice chance to see where you're at
- No edge cases on double or int values are gonna be tested
  - (NaN, infinity, etc.)

Review

`ArrayUtil.zeroInsert(int[] x)`

Exercise 1: Prefix Construction

Given are two strings `s` and `t` and an integer `n` with $n \geq 0$. Write a method that returns whether `s` is a concatenation of at most `n` prefixes of `t`.

Examples:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: The method should return `true`, because `"abc"` and `"ab"` are prefixes of `t` and `s` is a concatenation of `"abc"`, `"ab"`, `"abc"`.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: The method should return `false`, because `"bc"` is not a prefix of `t`.
- `s = "abab"`, `t = "abac"`, `n = 2`: The method should return `true`, because `"ab"` is a prefix of `t` and `s` is a concatenation of `"ab"`, `"ab"`.

Implement the method `isPrefixConstruction(String s, String t, int n)` in the class `PrefixConstruction`. The method has three arguments: the two strings `s` and `t` and the integer `n`. You may assume that n is greater than or equal to 0. You will find tests in the file "PrefixConstructionTest.java".

Tip: Solve the task recursively.

Tip: Check s = "ababa" t = "aba" n = 2 on paper.

Exercise 1: Prefix Construction

Greedy approach does not always work!

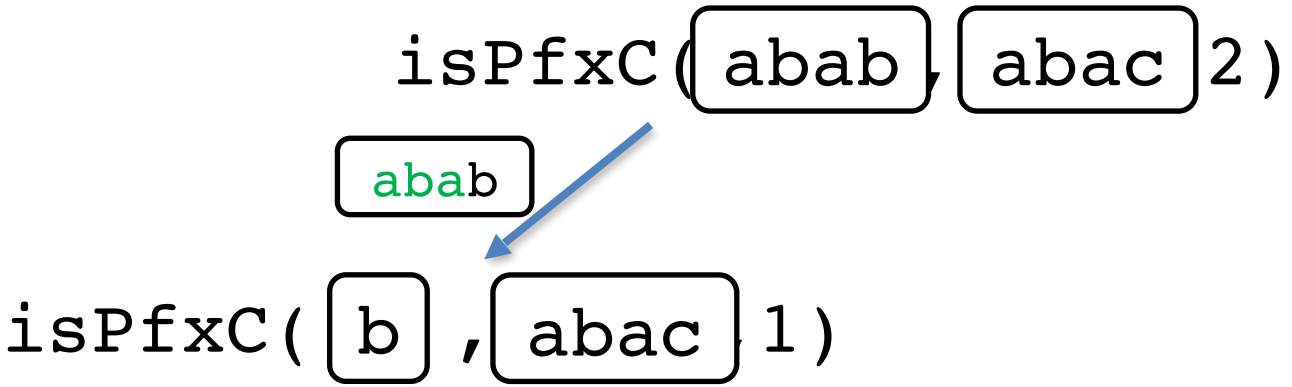
Solution: Exhaustive Search (Brute-Force)

```
s="abab", t="abac",
n=2
```

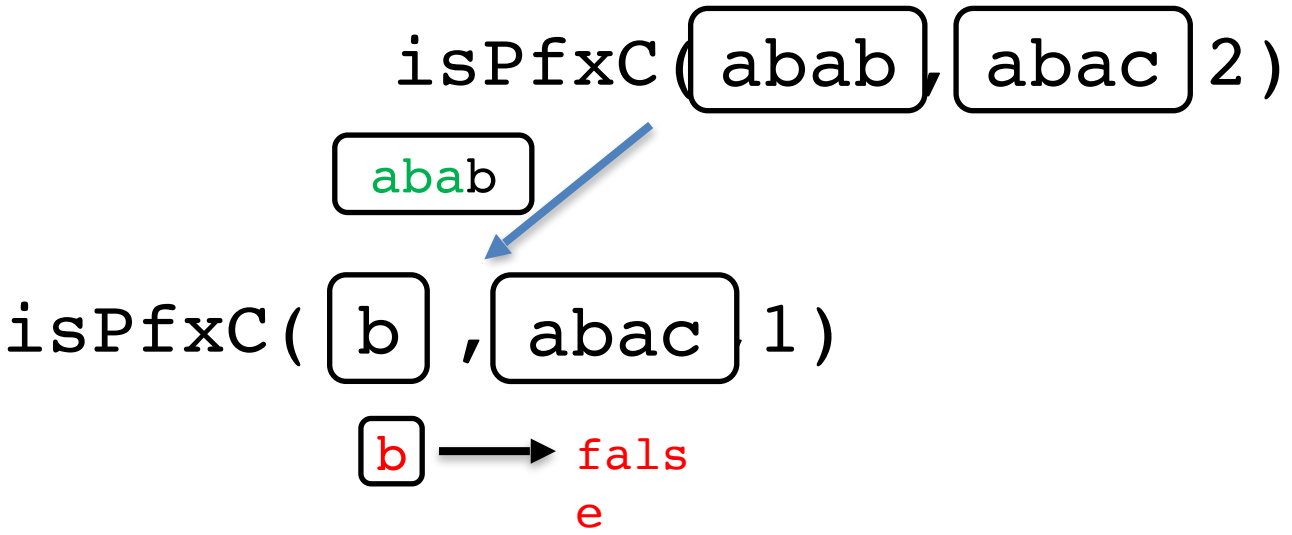
Exercise 1: Prefix Construction



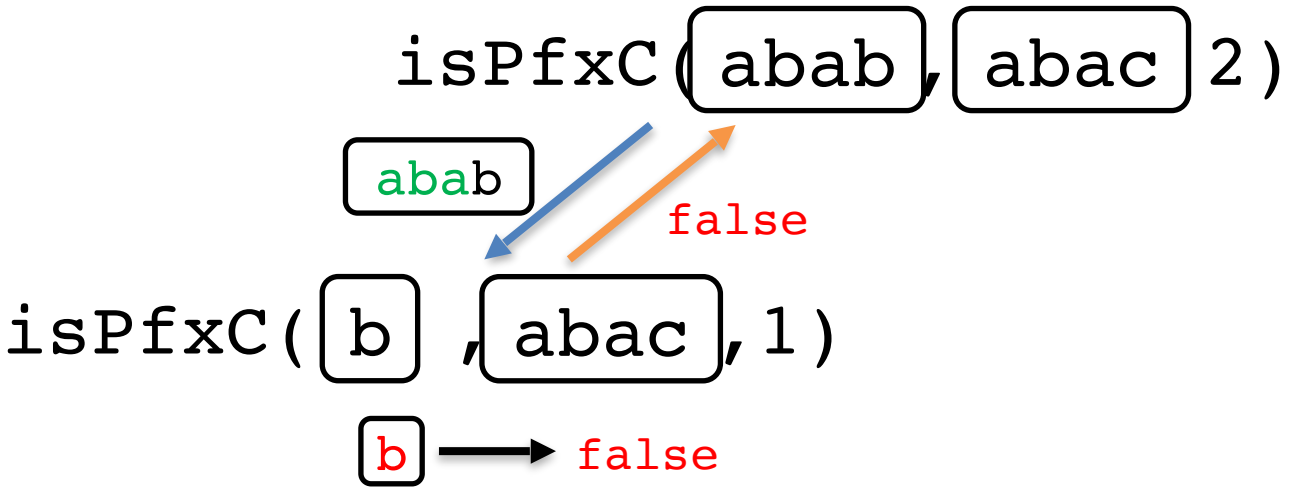
Exercise 1: Prefix Construction



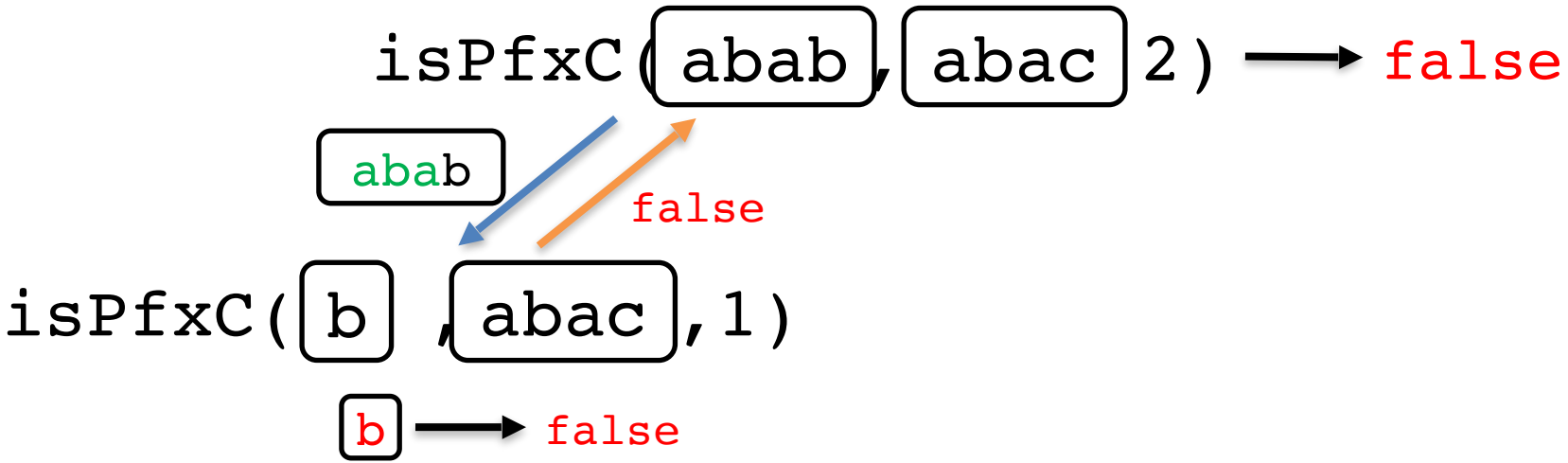
Exercise 1: Prefix Construction



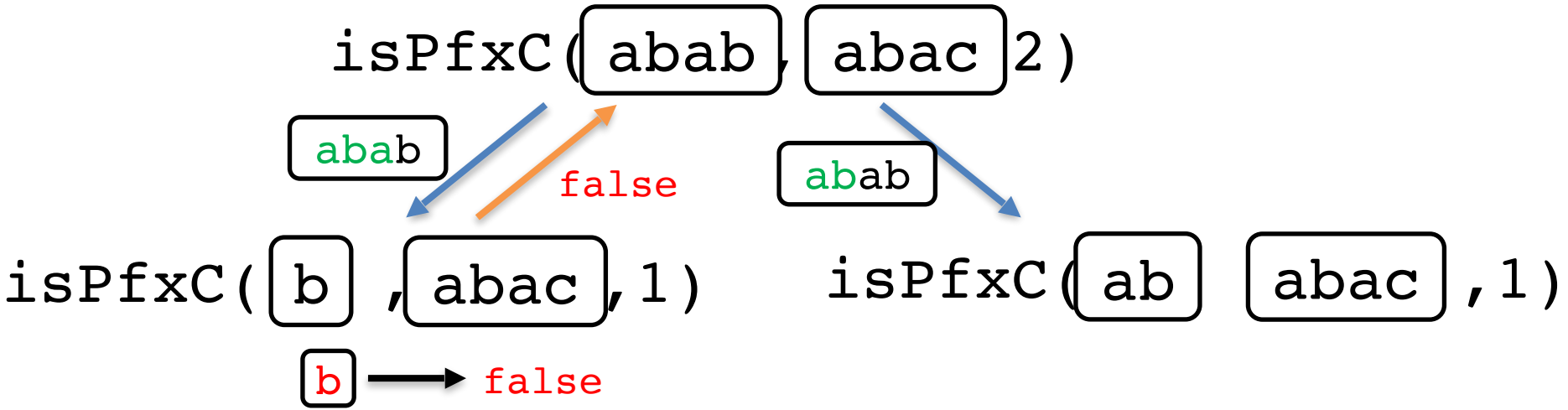
Exercise 1: Prefix Construction



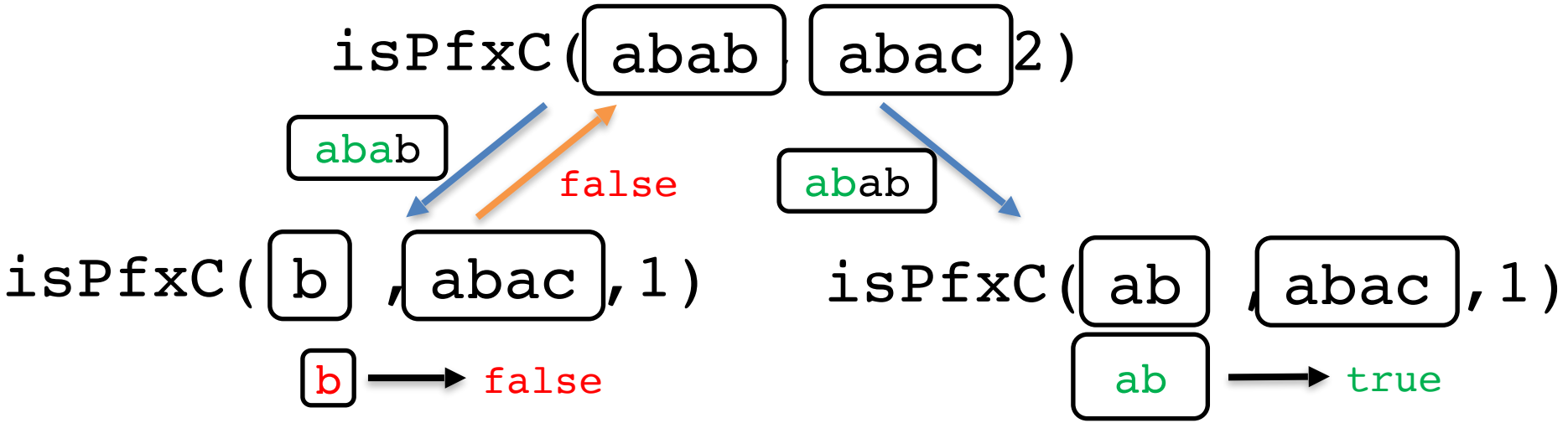
Exercise 1: Prefix Construction



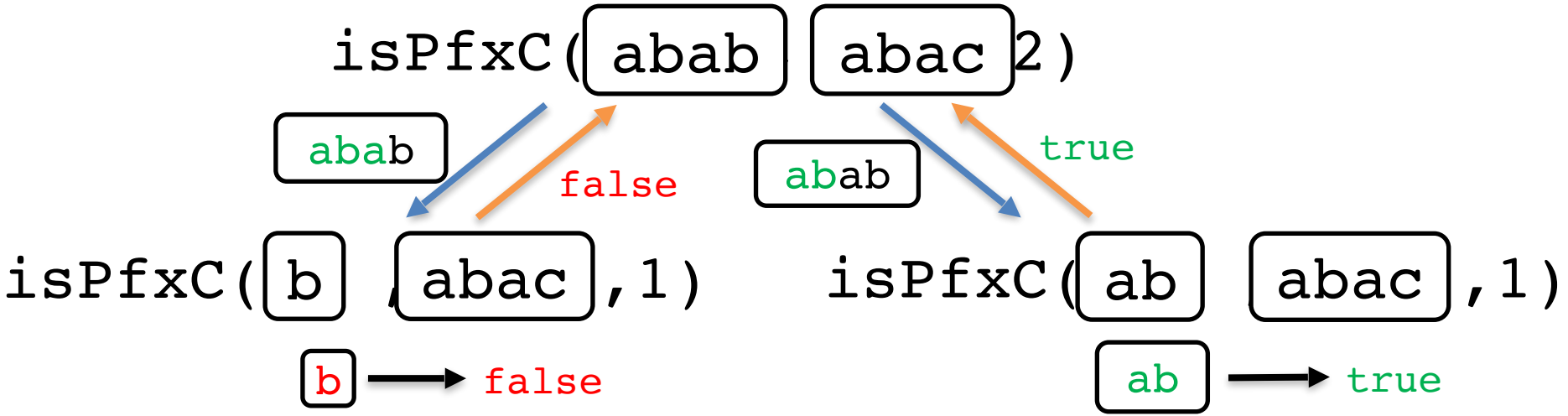
Exercise 1: Prefix Construction



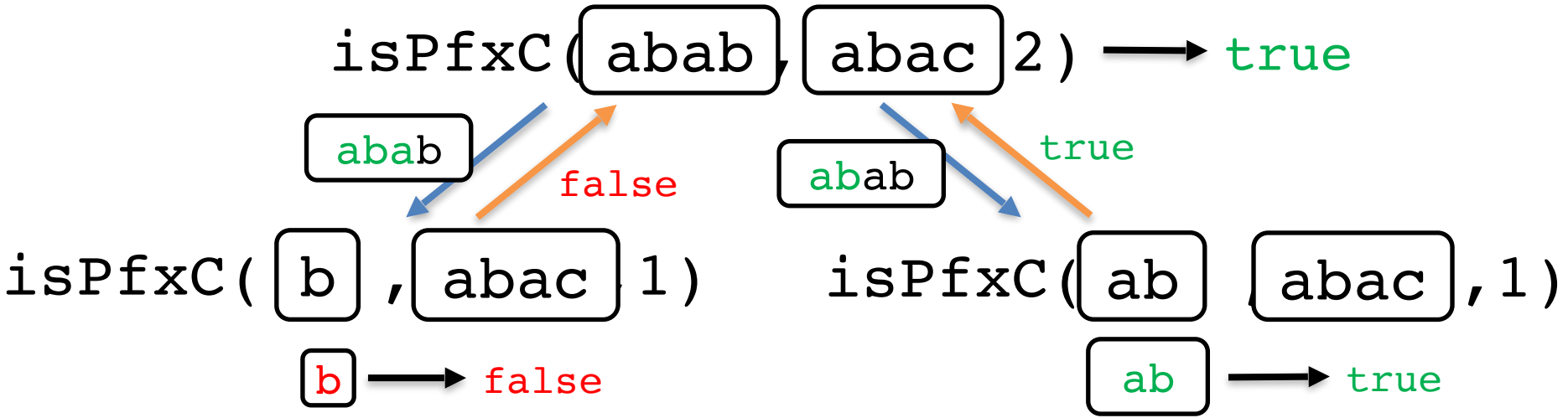
Exercise 1: Prefix Construction



Exercise 1: Prefix Construction



Exercise 1: Prefix Construction



```java
public class PrefixConstruction {

  public static boolean isPrefixConstruction(
      String s, String t, int n) {

    // Base case: nothing left to build → success
    if (s.isEmpty()) {
      return true;
    }

    // s is not empty, but no pieces left → failure
    if (n ⩽ 0) {
      return false;
    }

    // Brute force: try every prefix of t as the next piece
    for (int len = 1; len ⩽ t.length(); len++) {
      String prefix = t.substring(0, len);

      // Does s start with this prefix?
      // If yes: build the rest of s with one piece less
      if (s.startsWith(prefix)
          && isPrefixConstruction(s.substring(len), t, n - 1)) {
        return true;
      }
    }

    // No prefix worked → failure
    return false;
  }
}
```

Preview

Exercise 1: Triangle Matrix

The class `Triangle` allows the representation of $Z \times S$ triangular matrices (of `int` values). Z and S are always strictly greater than 1 (i.e. > 1). A $Z \times S$ triangular matrix has Z rows $X_0, X_1, \ldots, X_{Z-1}$, where row X_i has exactly $(i * (S-1)/(Z-1)) + 1$ elements. This expression is evaluated according to the rules for `int` expressions in Java. For a triangular matrix `d`, $\texttt{d}_{i,j}$ is the $(j+1)$-th element in the $(i+1)$-th row. $\texttt{d}_{0,0}$ is the first element in the first row (which always has exactly 1 element). Figure 1 shows examples of triangular matrices. Note that it is possible for two (consecutive) rows to have the same number of elements.

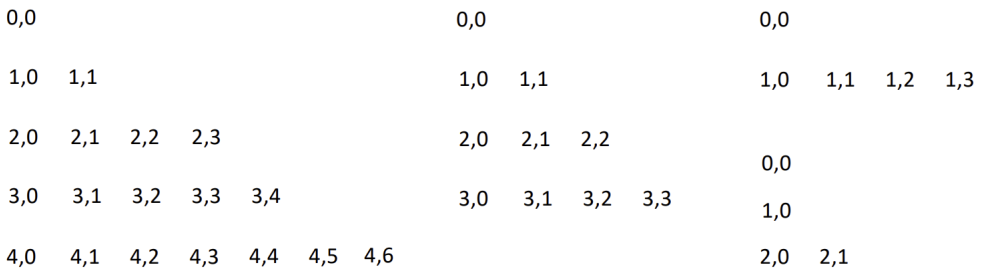


Figure 1: Examples of 5×7, 4×4, 2×4, and 3×2 triangular matrices.

Exercise 1: Triangle Matrix

In the file `Triangle.java`, you will find the class `Triangle` with a constructor `Triangle(int z, int s)` that creates a $z \times s$ triangular matrix. This constructor sets the values of all elements to 0. Complete this class so that the following methods are supported:

1. `int get(int i, int j)` returns the element $d_{i,j}$.
2. `void put(int i, int j, int value)` sets the element $d_{i,j}$ to the value `value`.
3. `int[] linear()` returns the elements in canonical order: first by increasing row index and, within each row, by increasing column index.
4. `void init(int[] data)` replaces the elements of `d` with the values in `data`. You may assume that `data` has exactly as many elements as `d`. The method sets the elements of `d` so that the sequence `d.init(data); int[] y = d.linear();` results in an array `y` for which `Arrays.equals(y, data)` yields the value `true`.
5. `void add(Triangle t)` adds, when called as `d.add(t)`, the value of $t_{i,j}$ to each element $d_{i,j}$ if $t_{i,j}$ exists. If $t_{i,j}$ does not exist, then $d_{i,j}$ remains unchanged.

Tests can be found in the file "TriangleTest.java". This task is an old exam task. The file "GradingTriangleTest.java" contains the tests that we used for grading in the exam. We recommend using these tests only once you think your solution is correct, so that you can see how you would have performed in an exam.

Exercise 5: Levels

Very important exercise!

Task 5: Levels

Implement the method `sumLevel(int[][] matrix, int level)` in the class `Levels`, which computes the sum of the elements of the specified level for a square matrix (parameter `matrix`) and a given "level" number (parameter `level`).
A level is defined by the layers of the matrix, counted from the inside outward. Level 0 corresponds to the center of the matrix. Each higher level describes the next outer border of the matrix.

In Figure 2, you will find two examples of matrices and their levels. In Figure 2a, for example, the innermost level, Level 0, consists of only one element. Figure 2b shows a matrix with three levels: the outer border of the matrix is Level 2, the middle border is Level 1, and Level 0 consists of the innermost four elements.

1 Level: 1	2 Level: 1	3 Level: 1
4 Level: 1	5 Level: 0	6 Level: 1
7 Level: 1	8 Level: 1	9 Level: 1

(a) 3 × 3 matrix

10 Level: 2	20 Level: 2	30 Level: 2	40 Level: 2	50 Level: 2	60 Level: 2
15 Level: 2	25 Level: 1	35 Level: 1	45 Level: 1	55 Level: 1	65 Level: 2
70 Level: 2	80 Level: 1	90 Level: 0	100 Level: 0	110 Level: 1	120 Level: 2
5 Level: 2	10 Level: 1	15 Level: 0	20 Level: 0	25 Level: 1	30 Level: 2
1 Level: 2	2 Level: 1	3 Level: 1	4 Level: 1	5 Level: 1	6 Level: 2
99 Level: 2	88 Level: 2	77 Level: 2	66 Level: 2	55 Level: 2	44 Level: 2

(b) 6 × 6 matrix

Figure 2: Examples of valid matrices and their levels.

Weakest Precondition with Branching

Examples

Weakest Precondition – With Branching

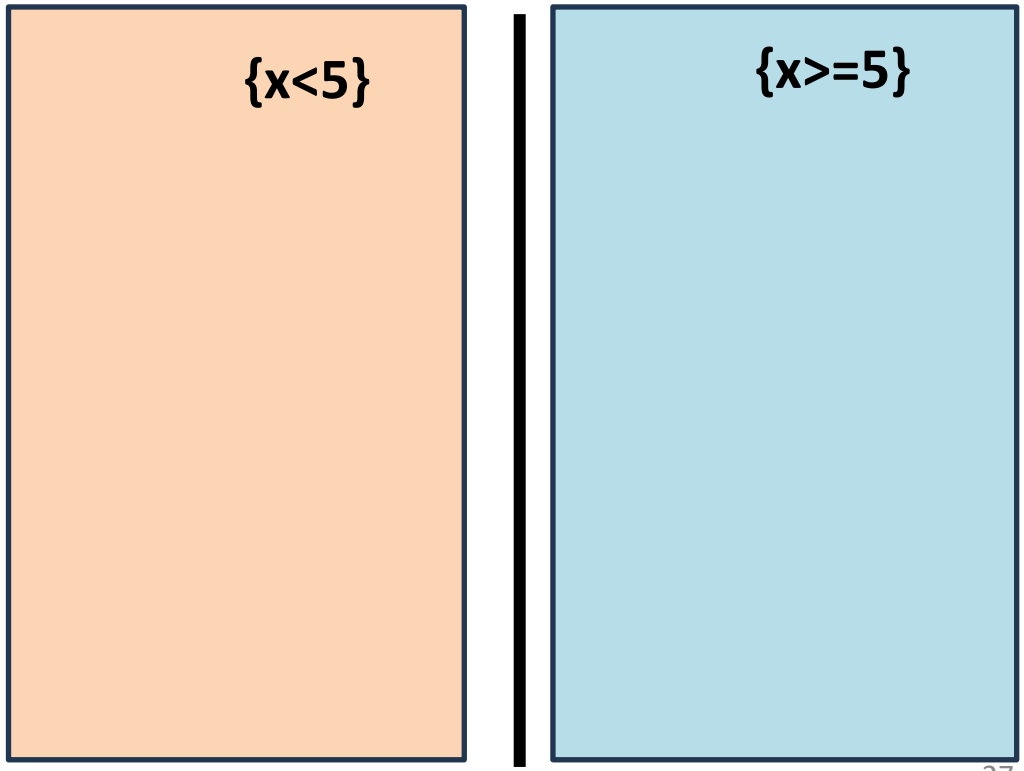
```
{              }
if (x<5) {
        y = x*x;
} else {
        y = x+1;
}
{y>9}
```

Weakest Precondition – With Branching

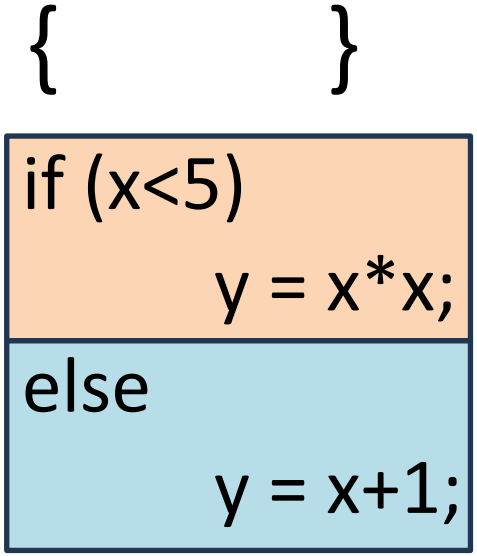
{ }

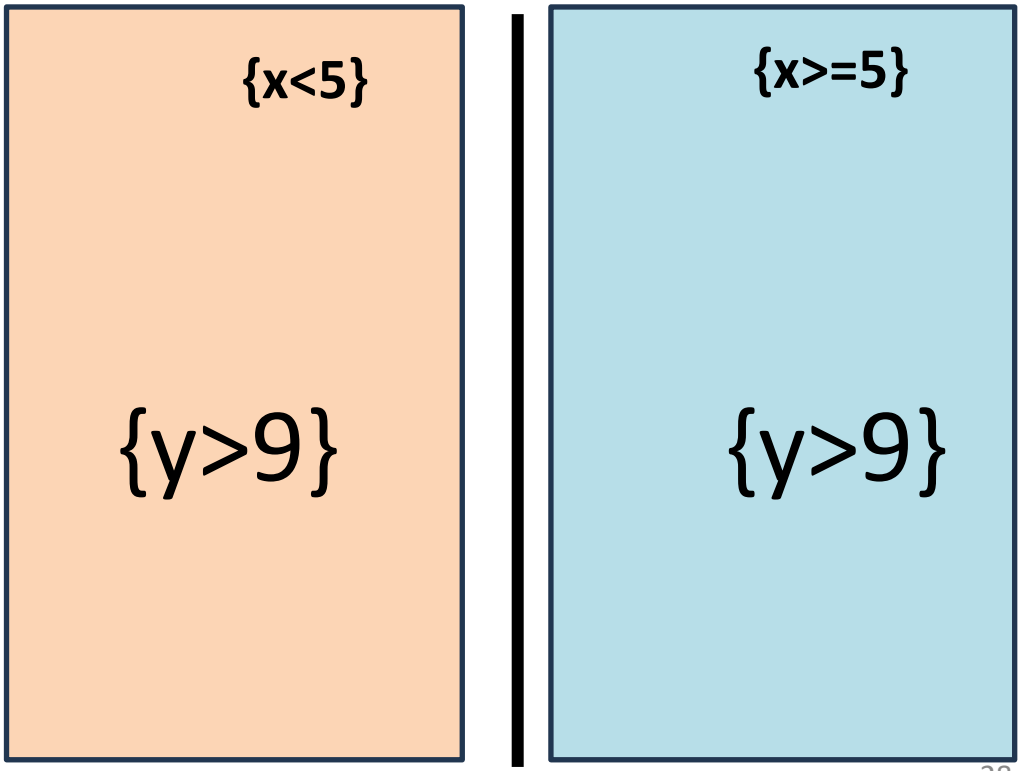
```
if (x<5)
        y = x*x;
else
        y = x+1;
```

{y>9}

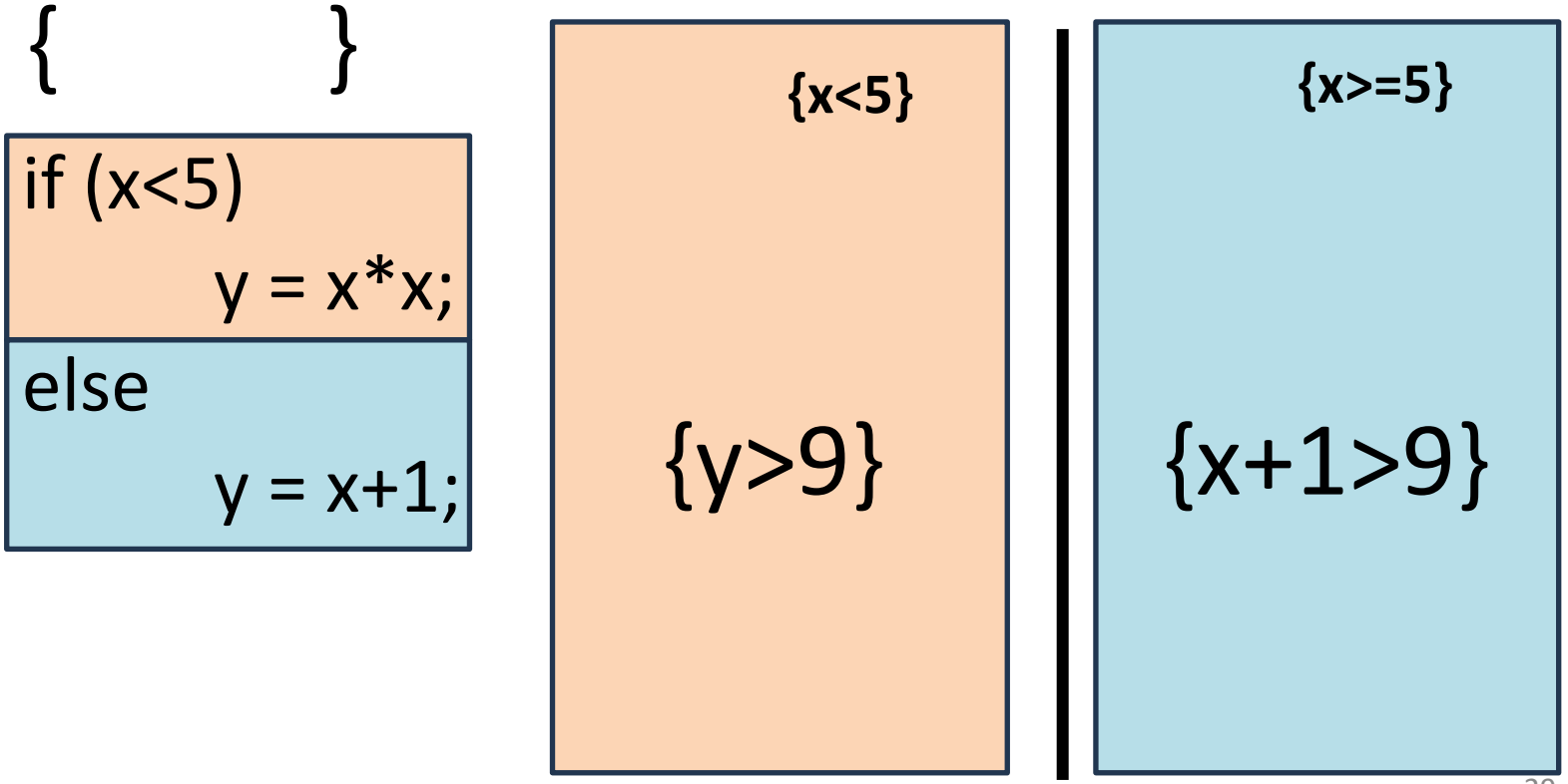


Weakest Precondition – With Branching

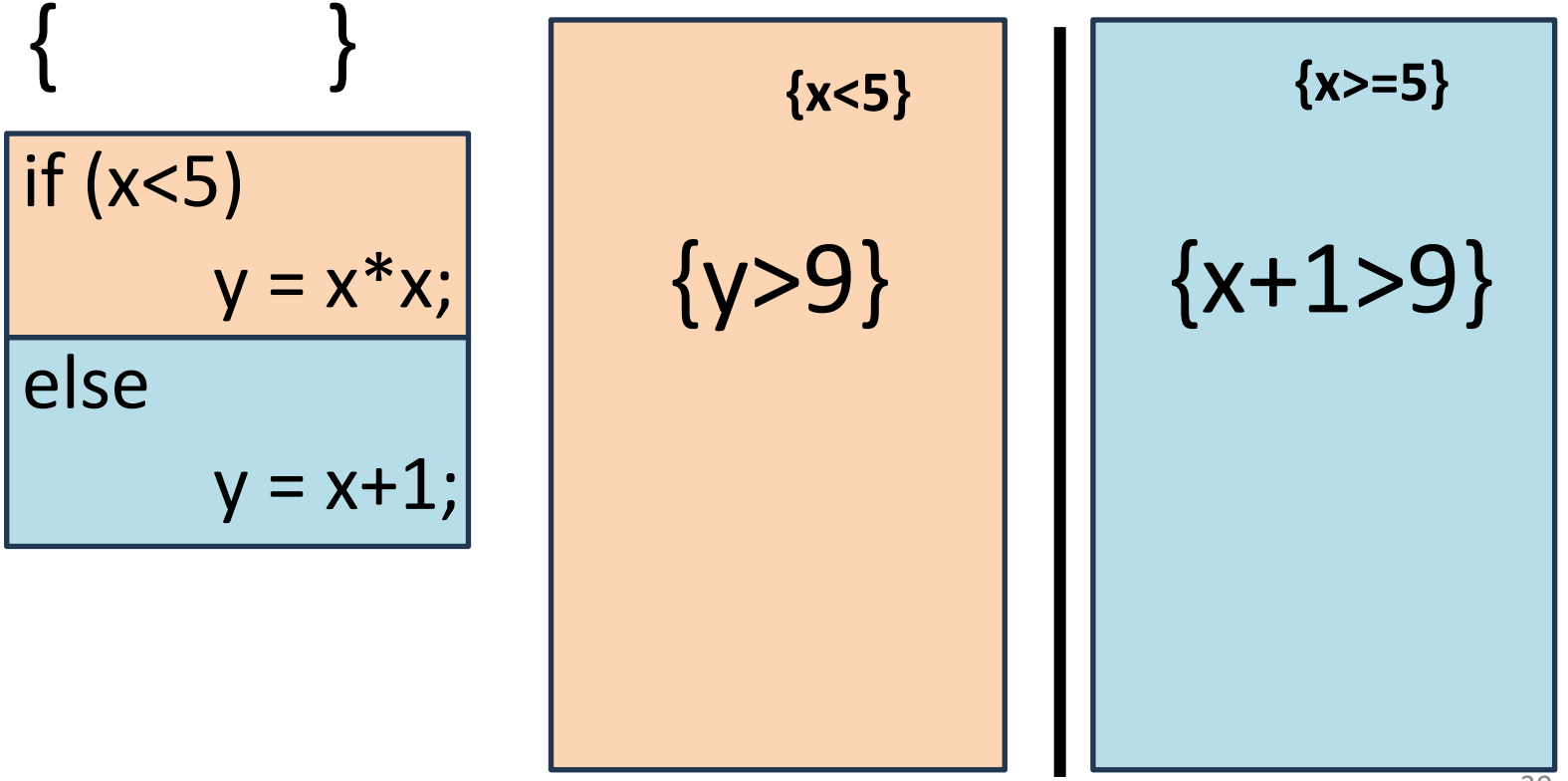




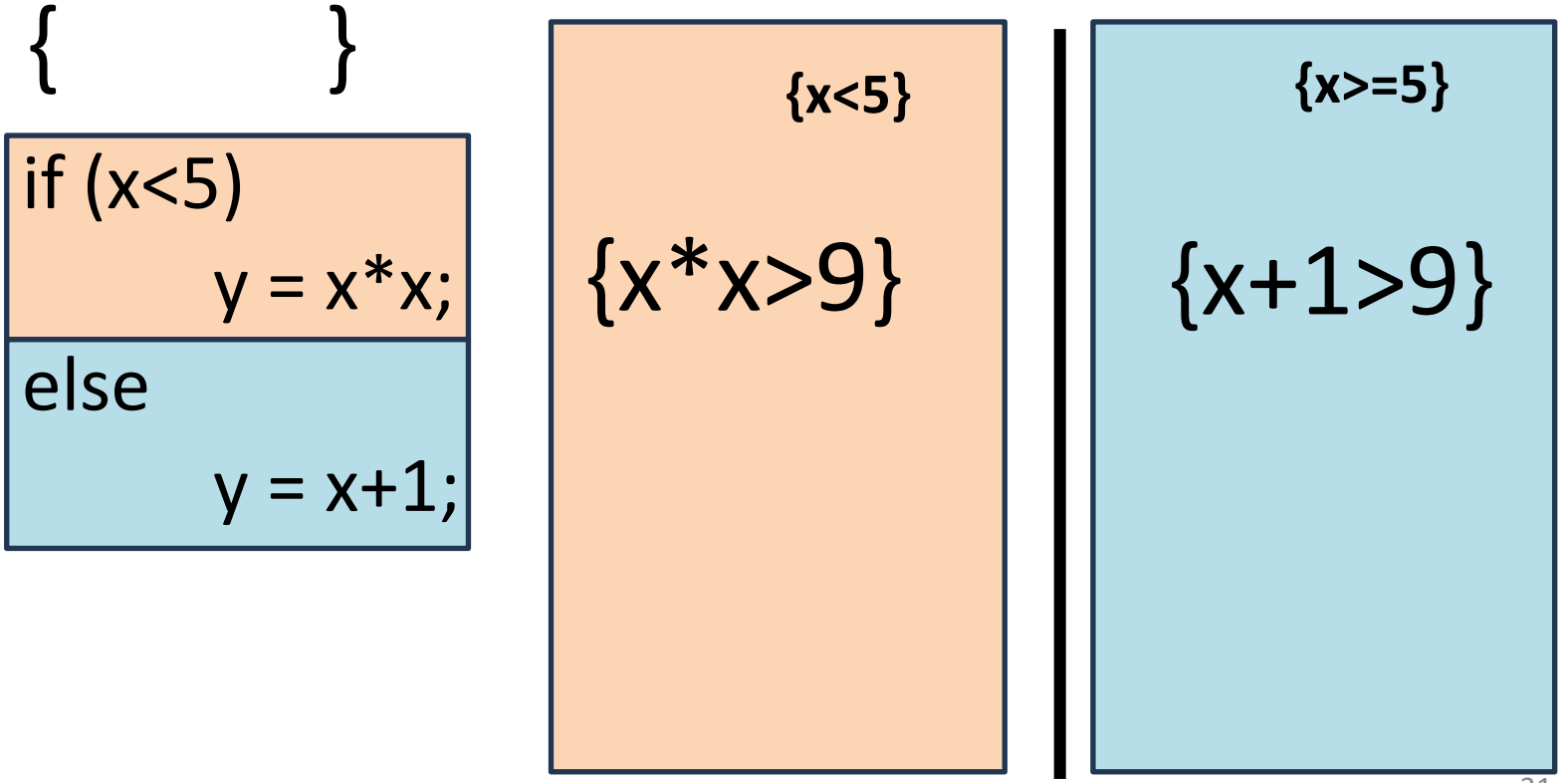
Weakest Precondition – With Branching



Weakest Precondition – With Branching



Weakest Precondition – With Branching



{(**x<5** && x*x>9) || (**x>=5** && x+1>9)}

{ }

```
if (x<5)
        y = x*x;
else
        y = x+1;
```

{x<5}

{x*x>9}

||

{x>=5}

{x+1>9}

Weakest Precondition - Exam Example

```
{              }
 if (x > y)
        z = x – y;
 else
        z = y – x;

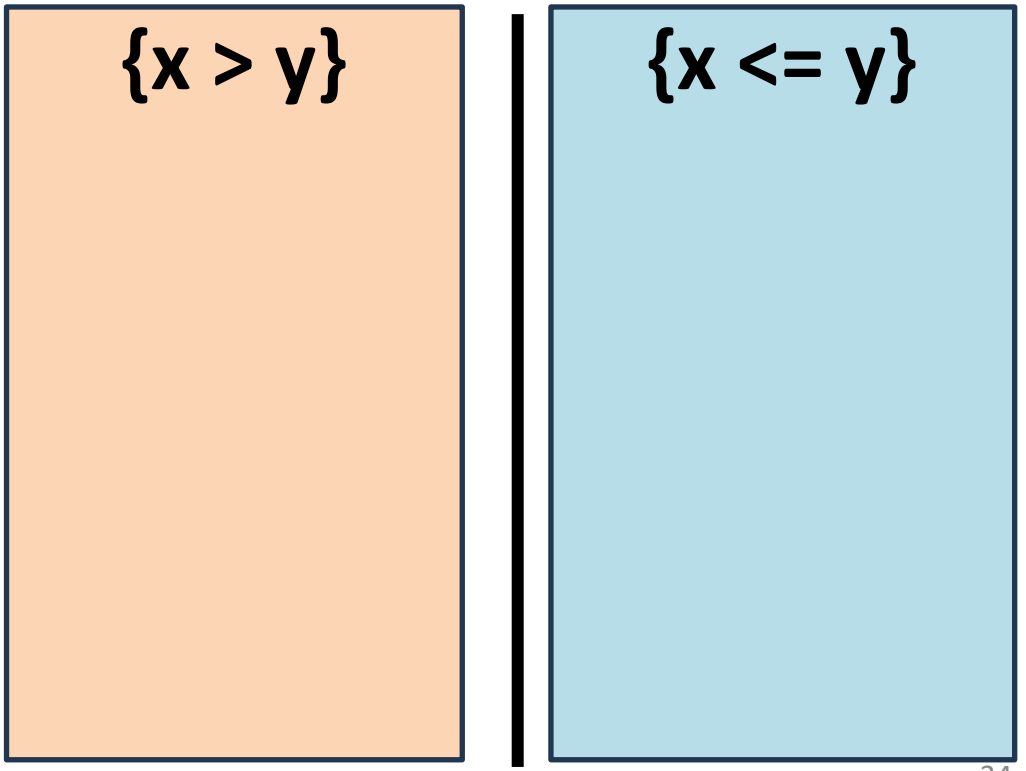
{  z > 0  }
```

Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

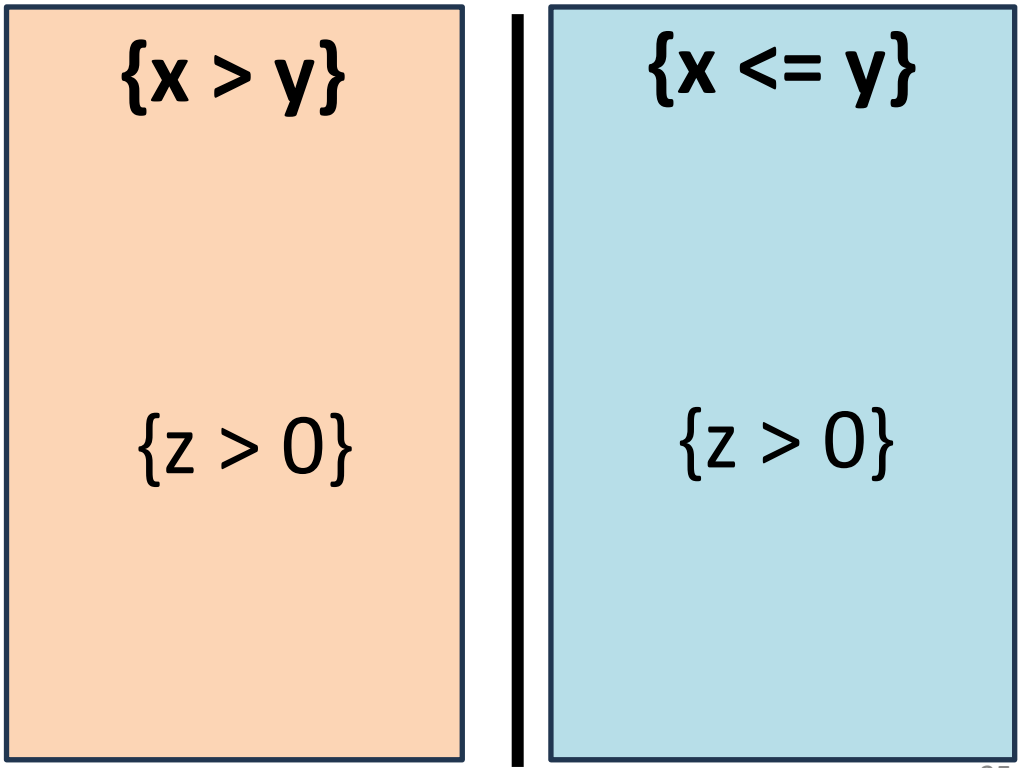


Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

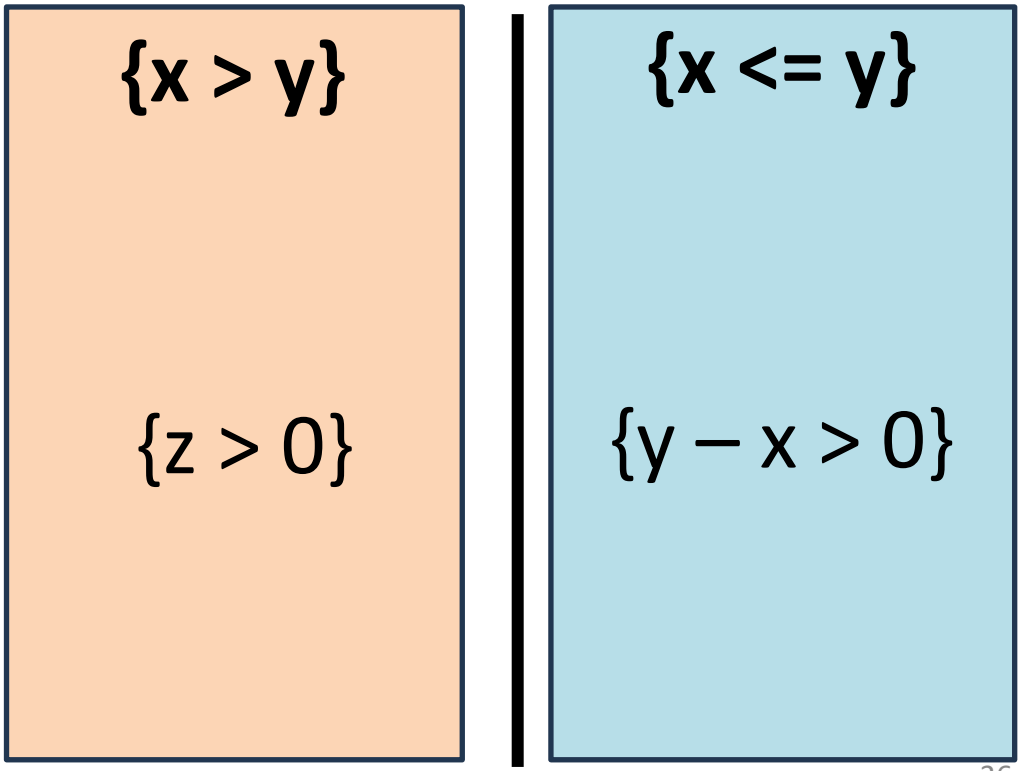


Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

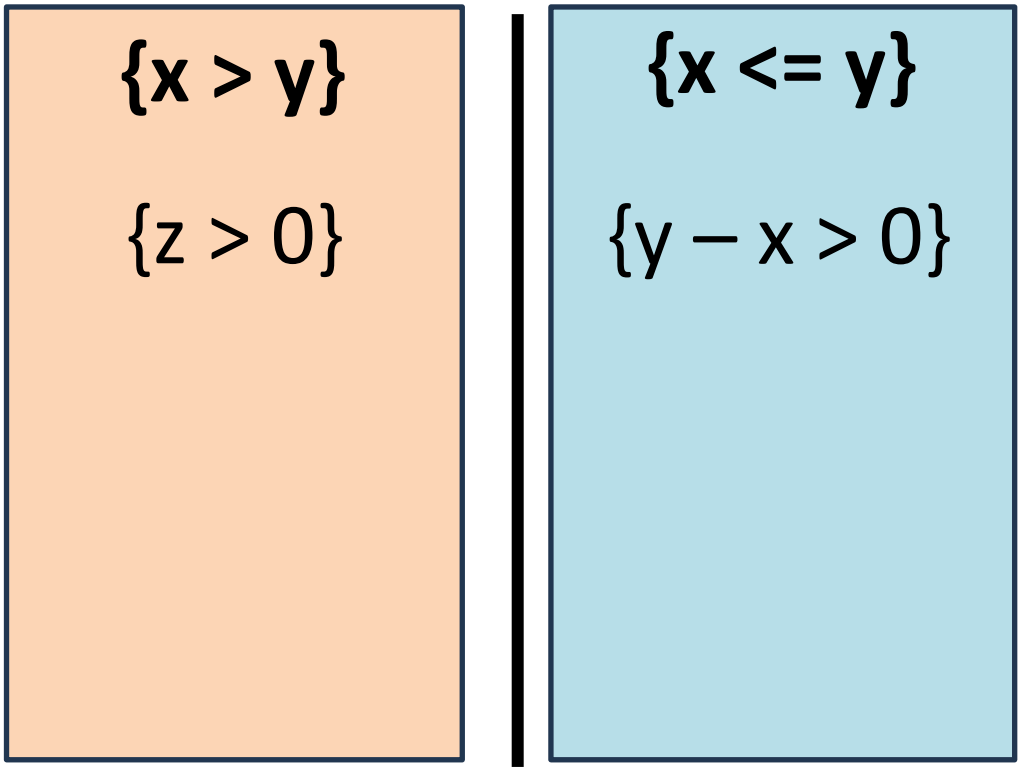


Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

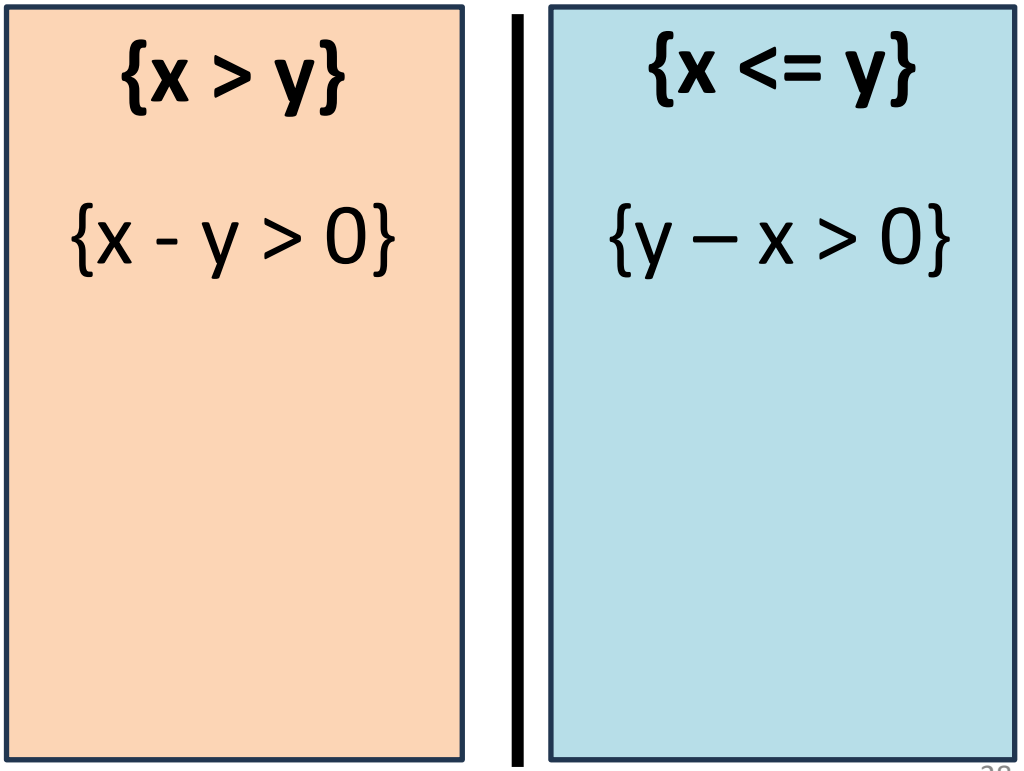


Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

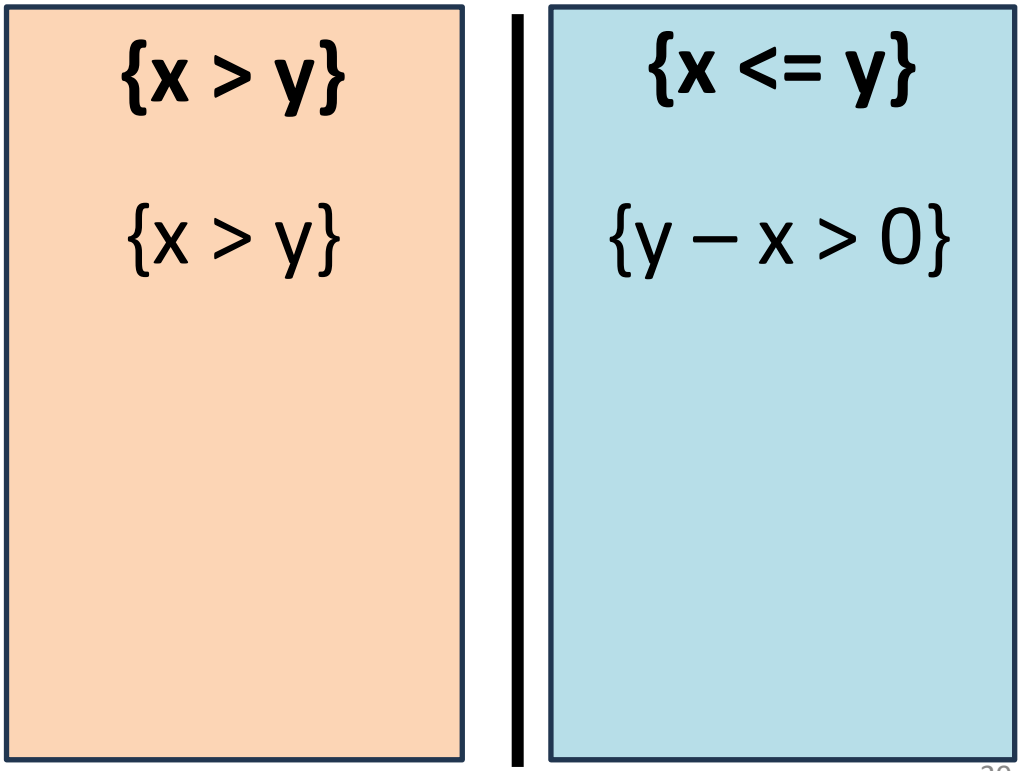


Weakest Precondition - Exam Example

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }



{((**x > y**) && (x > y)) || (**(x <= y)** &&(y > x))}

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

{x > y}

{x > y}

{x <= y}

{y > x}

Weakest Precondition - Example

```
{          }

  if (x >= y){
    y = x
  }

{y >= x}
```

Weakest Precondition with Branching

Theory

Weakest Precondition

- The weakest precondition is the weakest precondition that implies the postcondition.
  - If the postcondition is { true }, then { true } is the weakest precondition. Everything implies the postcondition, including in particular the weakest condition true.
  - If the postcondition is { false }, then { false } is the weakest precondition. Only { false } implies the postcondition, so it is the weakest (and only) precondition.
- The rules introduced for backward reasoning directly produce the weakest preconditions.

If Statements and Assertions

Goal: rule for triples {P} if (b) S1 else S2 {Q}

Observations

- Execution of S1 when b holds
- Execution of S2 when ¬b holds
- P holds before execution in both cases
- Q must hold afterward in both cases

```
{P}
 if (b) {
   S1;
 } else {
   S2;
 }
{Q}
```

Hoare logic rule for if statements

The triple {P} if (b) S1 else S2 {Q} is valid

1. exactly when
2. {P ∧ b} S1 {Q} is valid
3. {P ∧ ¬b} S2 {Q} is valid

Procedure for if statements

- **Situation: decide whether {P} if (b) S1 else S2 {Q} is valid**
- **Recommended approach:**
  1. Apply known rules as shown on the right (to S_1 and S_2)
  2. Show necessary implications
     1. $P \wedge b \implies R_1$
     2. $P \wedge \neg b \implies R_2$
-

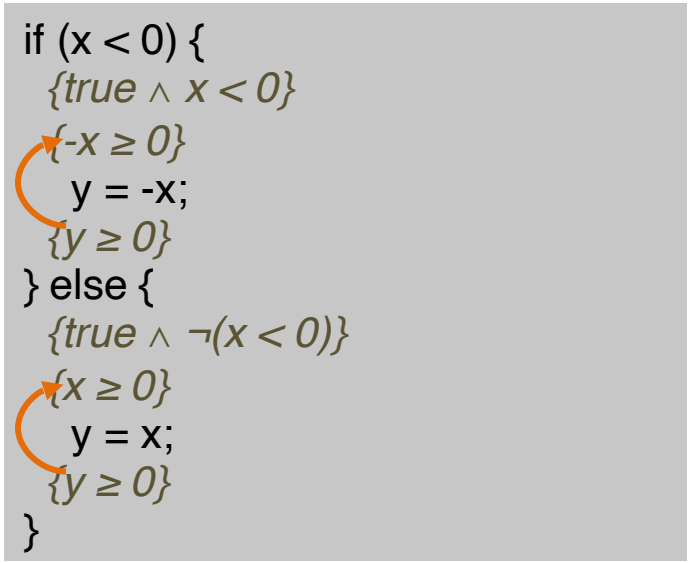
```
if (b) {
  {P ∧ b}
    {R1}
    S1;
  {Q}
} else {
  {P ∧ ¬b}
    {R2}
    S2;
  {Q}
}
```

Example 1: Valid Triple

To show: validity of

$\{true\}$ if (x < 0) { y = -x; } else { y = x;} $\{y \geq 0\}$

1. Apply rules:



2. Show implications:

1. $(x < 0) \Longrightarrow (-x \geq 0)$ **P**
2. $(x \geq 0) \Longrightarrow (x \geq 0)$ **P**

Example 2: Invalid Triple

To show: validity of

Changed

$\{true\}$ if (x < 0) { y = -x; } else { y = x;} $\{y > 0\}$

1. Apply rules:

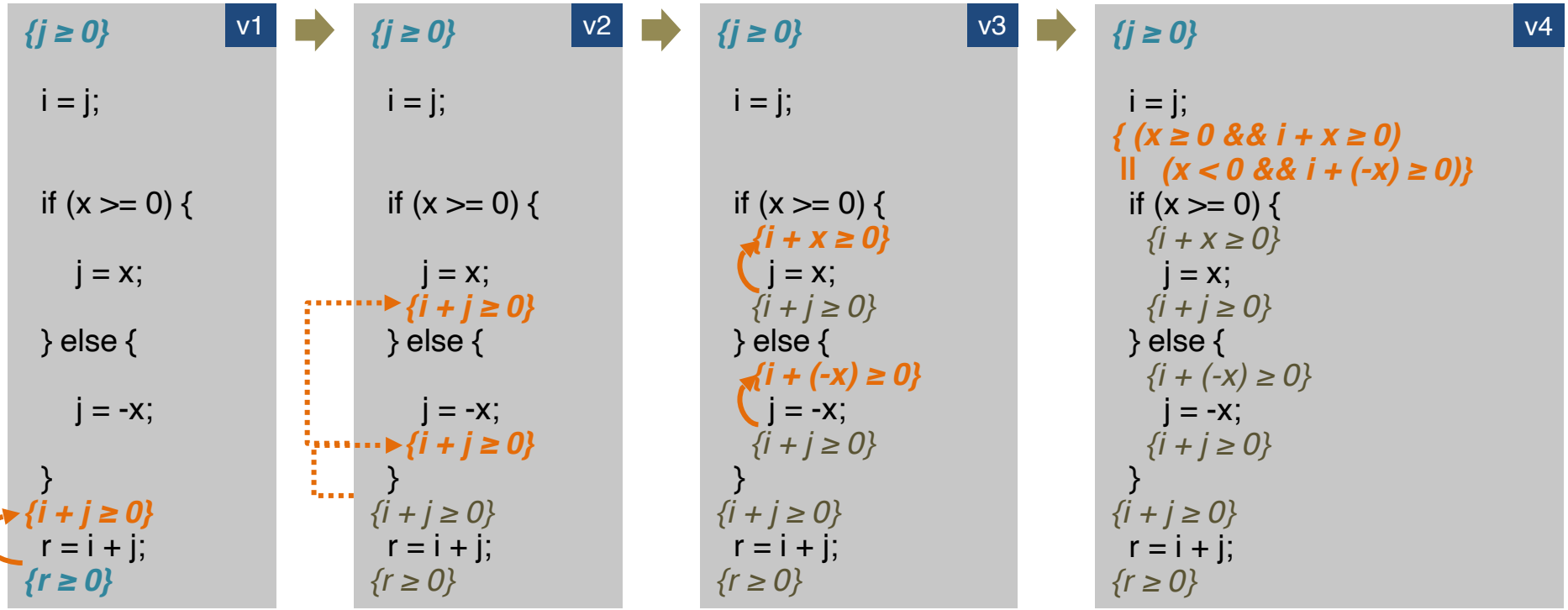
```
if (x < 0) {
  {true ∧ x < 0}
  {-x > 0}
    y = -x;
  {y > 0}
} else {
  {true ∧ ¬(x < 0)}
  {x > 0}
    y = x;
  {y > 0}
}
```

Corresponding differences

2. Show implications:
   1. $(x < 0) \Longrightarrow (-x > 0)$ **P**
   2. $(x \geq 0) \Longrightarrow (x > 0)$ **O**

No longer holds (counterexample: *x == 0*)

Example Step by Step



v4

```
{j ≥ 0}
{   (x ≥ 0 && j + x ≥ 0)
 ||  (x < 0 && j + (-x) ≥ 0) }
 i = j;
{   (x ≥ 0 ⟹ i + x ≥ 0)
 ∧  (x < 0 ⟹ i + (-x) ≥ 0) }
 if (x >= 0) {
   {i + x ≥ 0}
    j = x;
   {i + j ≥ 0}
 } else {
   {i + (-x) ≥ 0}
    j = -x;
   {i + j ≥ 0}
 }
{i + j ≥ 0}
 r = i + j;
{r ≥ 0}
```

v5

Then still to show:

$(j \geq 0)$

$\Longrightarrow$

$(\quad (x \geq 0 \;\&\&\; j + x \geq 0)$
$\;\;\|\quad (x < 0 \;\&\&\; j + (-x) \geq 0))$

case distinction (two conjuncts):

1. If j ≥ 0 and x ≥ 0, then j + x ≥ 0 **P**
2. If $j \geq 0$ and $x < 0$, then $j + (-x) \geq 0$ **P**

Hoare Triple

Hoare Triple – Exam Example HS18

```
{ b > c }
 if (x > b) {
        a = x;
} else {
        a = b;
}
{ a > c }
```

Hoare Triple – Exam Example HS18

```
{ true }
 if (x > y) {
        y = x;
} else {
        y = -x;
}
{ y >= x}
```

Hoare Triple – Exam Example HS18

$$\{ x > 0 \}$$

```
y = x * x;
z = y / 2;
```

$$\{ z > 0 \}$$

Hoare Logic FAQ

- Do we have to simplify our expressions?
  - As long as the solution is logically equivalent, it is correct.

- Do we have to show our derivation?
  - As long as this is not explicitly asked, it is not necessary.

- Do counterexamples have to contain all variables if only one of them is relevant?
  - If, for example, y != 0 always means that the postcondition does not follow from a precondition, then this is sufficient.

Permutation Check

Example Exercise

Given the integer arrays s and t. Check whether t is a permutation of s.

Solution 1: generate all permutations of s and check whether t is one of them.

Example Exercise

Given the integer arrays s and t. Check whether t is a permutation of s.

Solution 1: generate all permutations of s and check whether t is one of them.

```
Exception in thread "main" java.lang.OutOfMemoryError Create breakpoint : Java heap space
    at java.base/java.util.Arrays.copyOf(Arrays.java:3509)
    at java.base/java.util.Arrays.copyOf(Arrays.java:3478)
    at java.base/java.util.ArrayList.grow(ArrayList.java:238)
    at java.base/java.util.ArrayList.grow(ArrayList.java:245)
    at java.base/java.util.ArrayList.add(ArrayList.java:484)
    at java.base/java.util.ArrayList.add(ArrayList.java:497)
```

Out of Memory?

Java receives a limited amount of memory for executing a program.

In the previous solution we generate O(n!) permutations, all of which require memory.

At some point there is not enough memory anymore and the program terminates with a runtime error.

Example Exercise

Given the integer arrays s and t. Check whether t is a permutation of s.

Solution 2: generate all permutations of s one after another and check immediately after creating a permutation whether it equals t.



Example Exercise

Given the integer arrays s and t. Check whether t is a permutation of s.

Solution 3: check whether s and t have the same length and contain the same elements the same number of times.

Kahoot!

Weakest Precondition - Example

{ }

x = y + z;

y = y - 5;

{y < 0}

Weakest Precondition– Exam Example

```
{                    }

  w = a

  x = w + b;

  y = x * 2;

{y > 0 && b > 10}
```

Weakest Precondition– Exam Example

{ }

p = 3 * q

p = p + 1;

{p > 15}

Towers of Hanoi

Towers of Hanoi - Description

The Towers of Hanoi problem consists of three rods and a number of differently sized disks, which are initially stacked on one rod, with the largest disk at the bottom and the smallest at the top. The goal is to move all disks from a source rod to a target rod, where only one disk may be moved at a time and a larger disk may never be placed on a smaller one.

Steps

1. What is the simplest possible input?

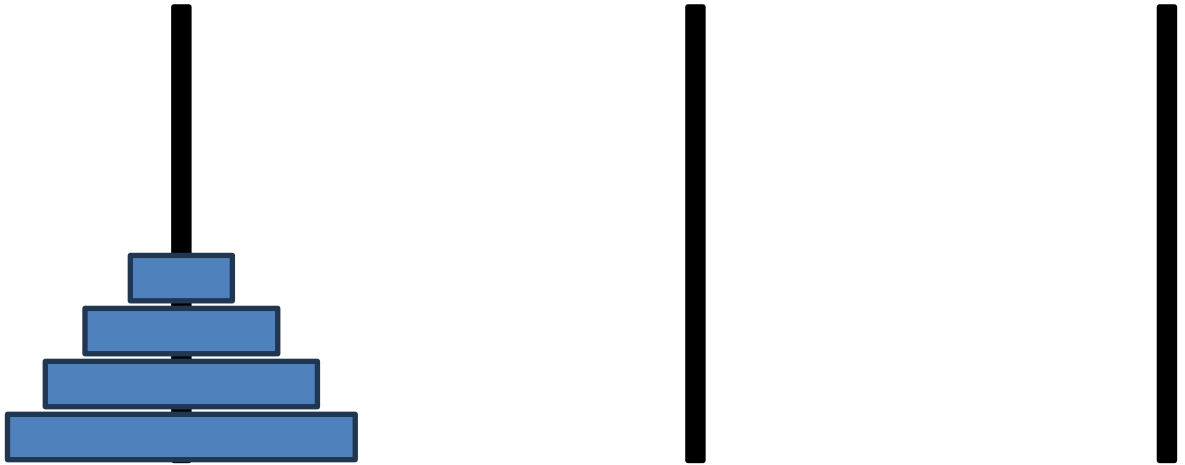
   -> Base Case

2. Try examples and visualize them
3. Derive larger examples from smaller examples
4. Generalize the pattern

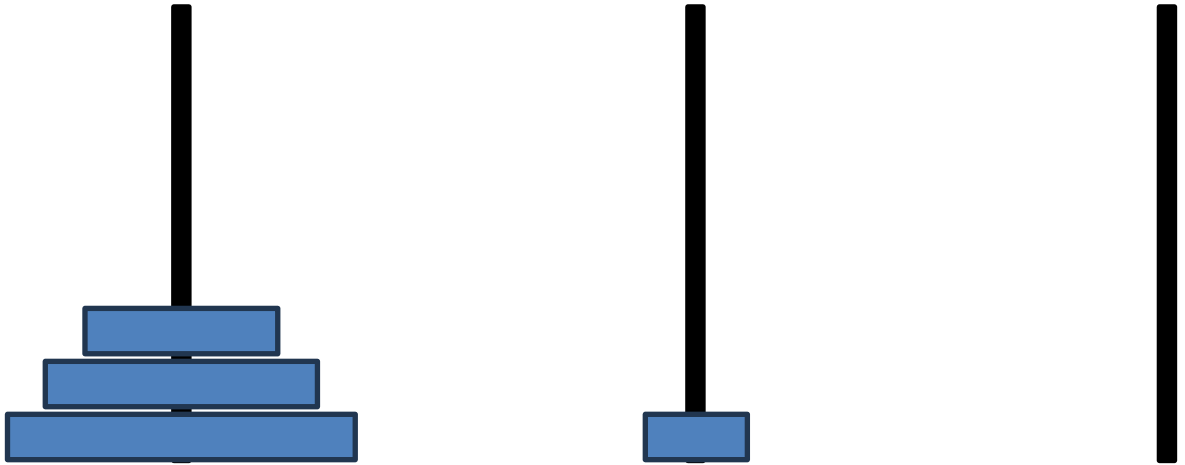
   -> recursive step

5. Write the code

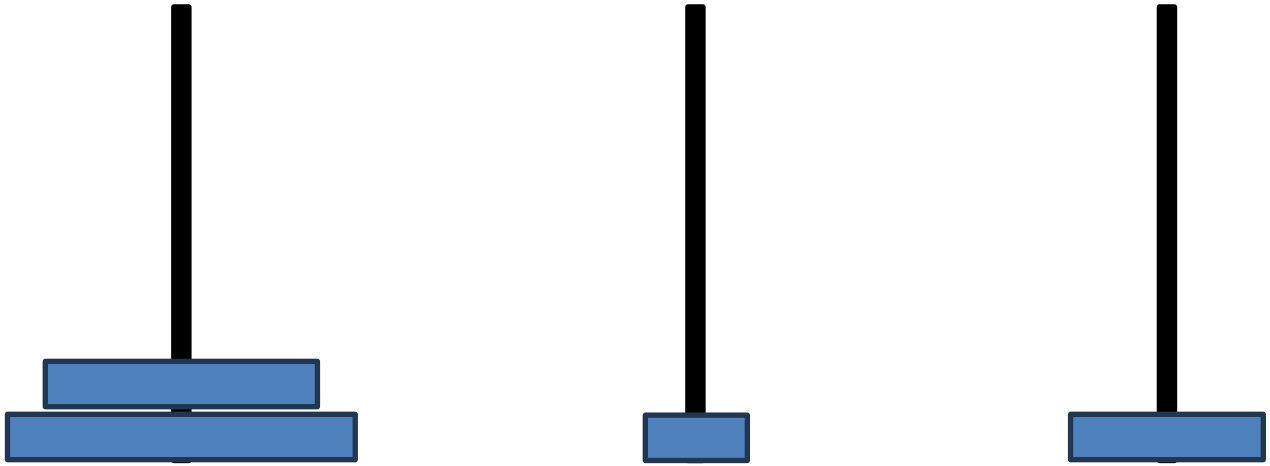
Towers of Hanoi - Example



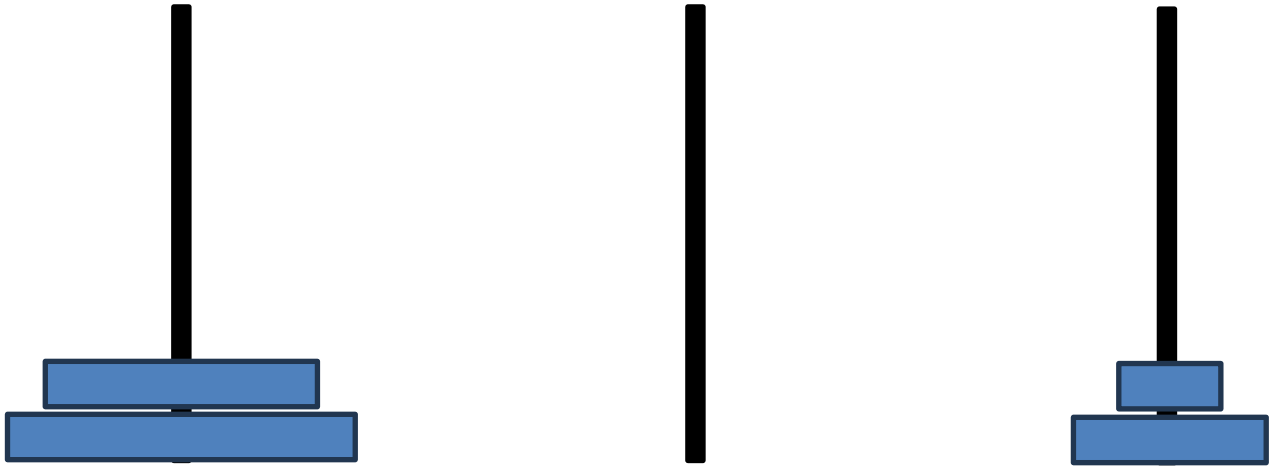
Towers of Hanoi - Example



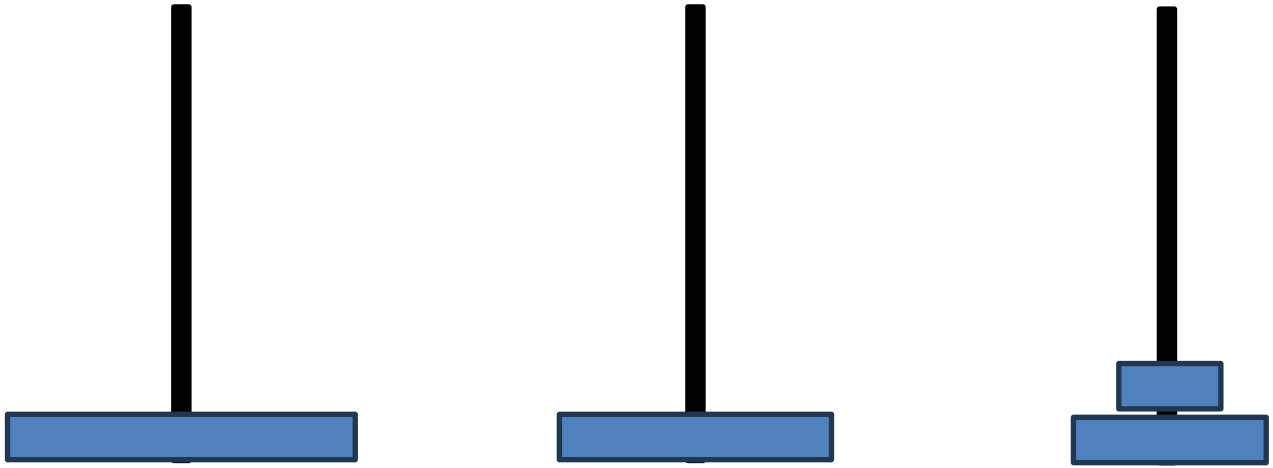
Towers of Hanoi - Example



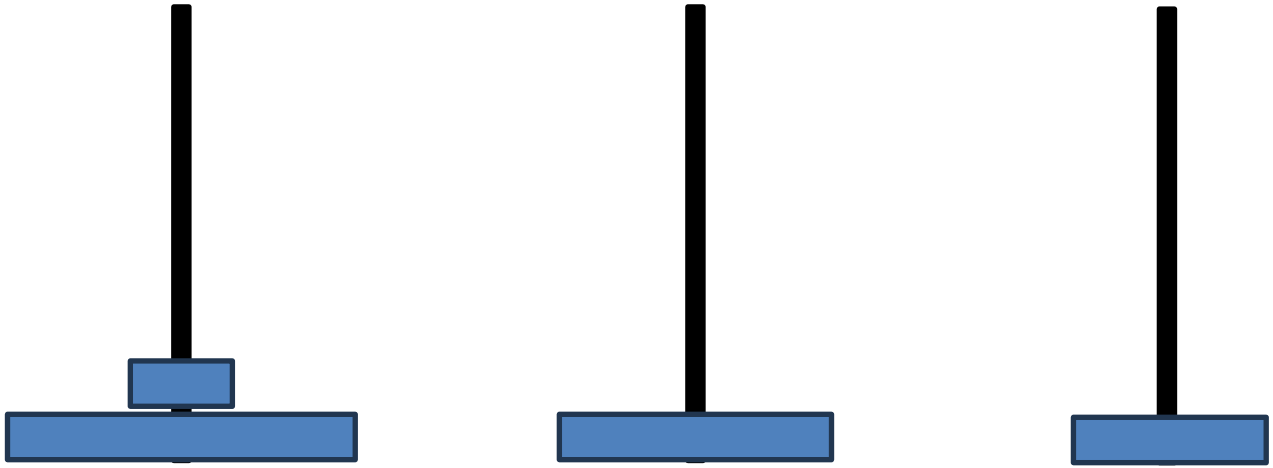
Towers of Hanoi - Example



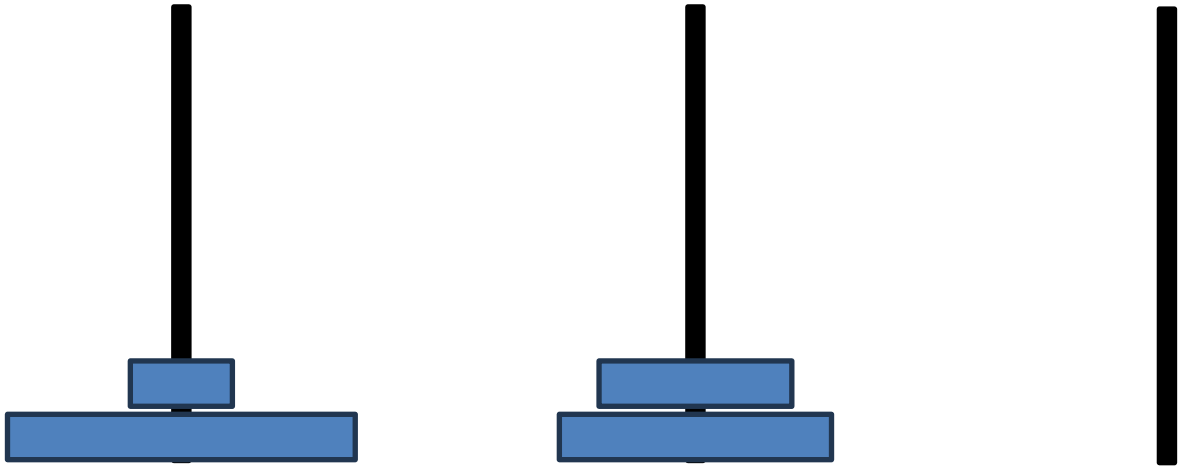
Towers of Hanoi - Example



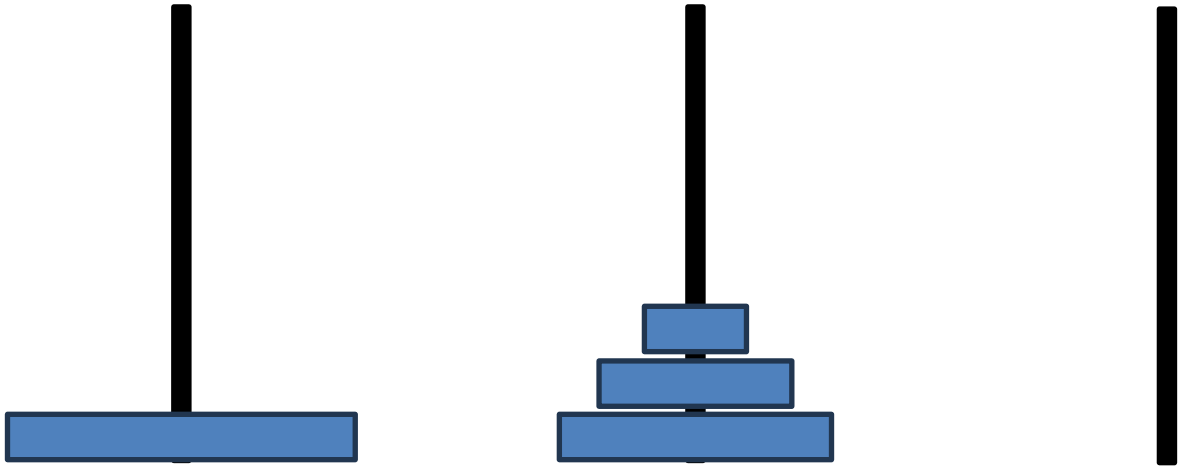
Towers of Hanoi - Example



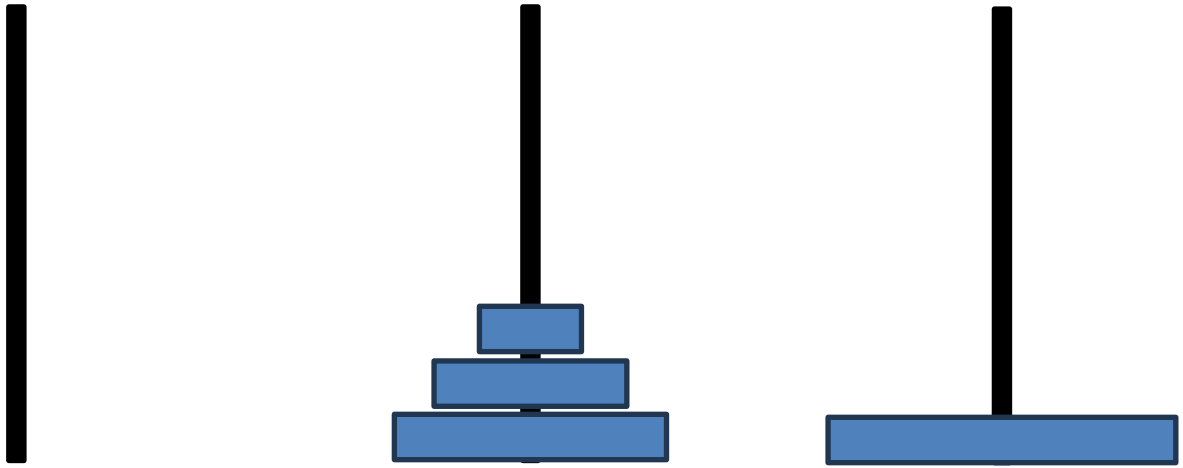
Towers of Hanoi - Example



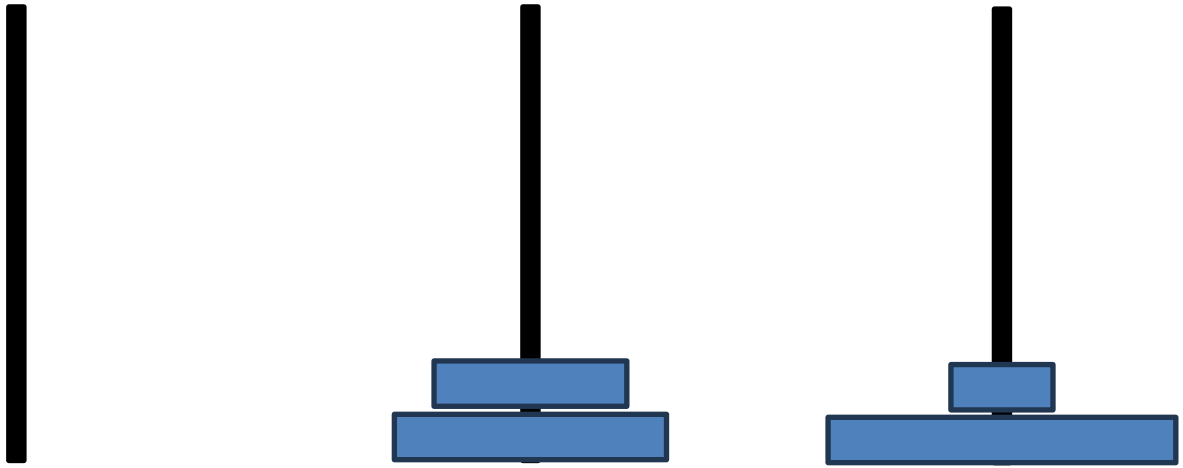
Towers of Hanoi - Example



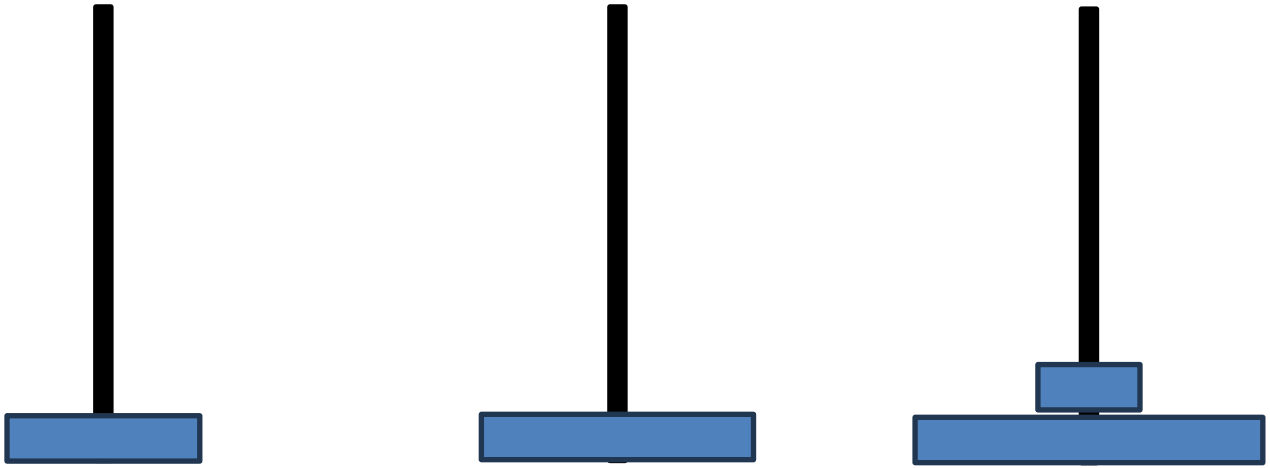
Towers of Hanoi - Example



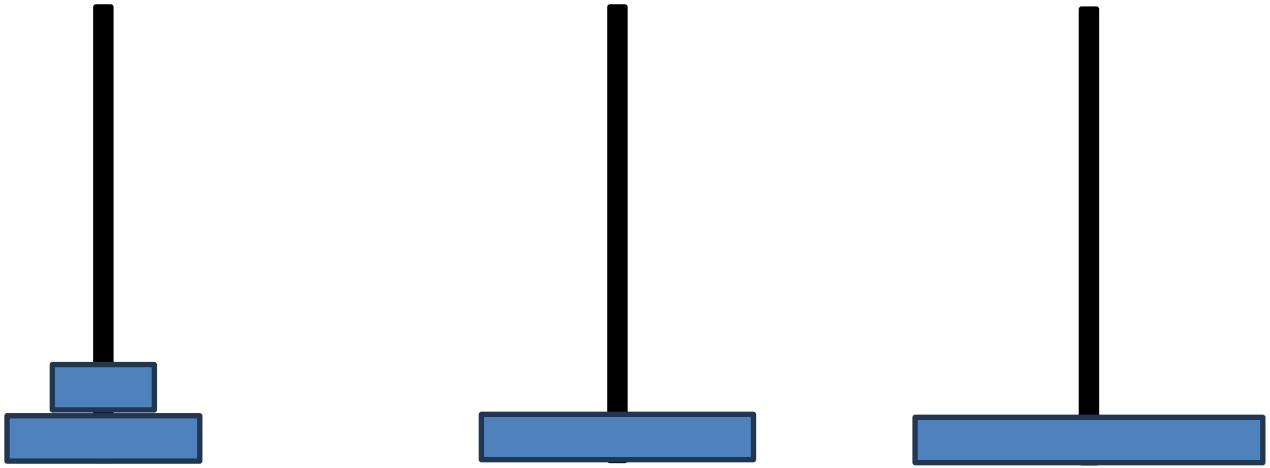
Towers of Hanoi - Example



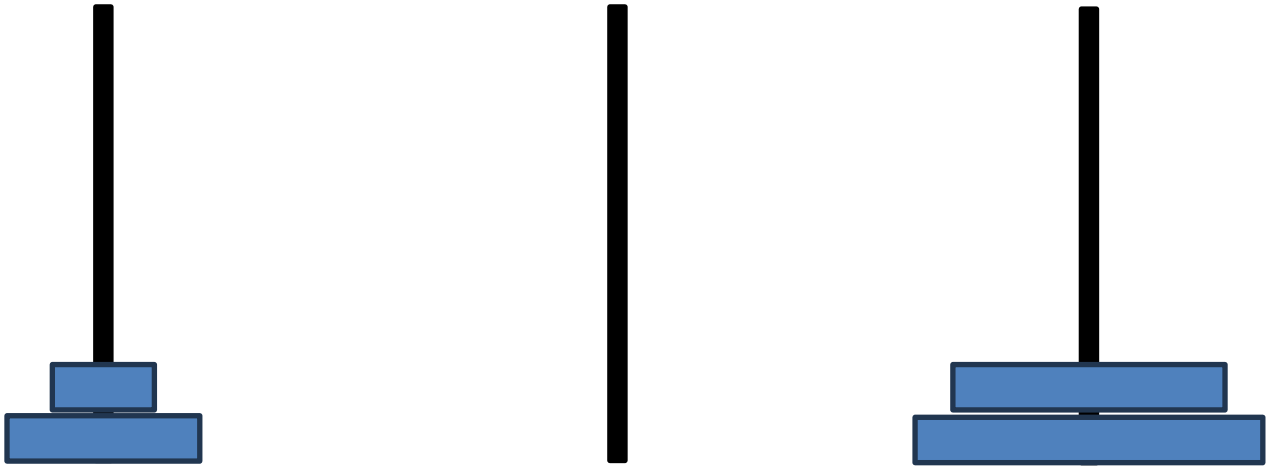
Towers of Hanoi - Example



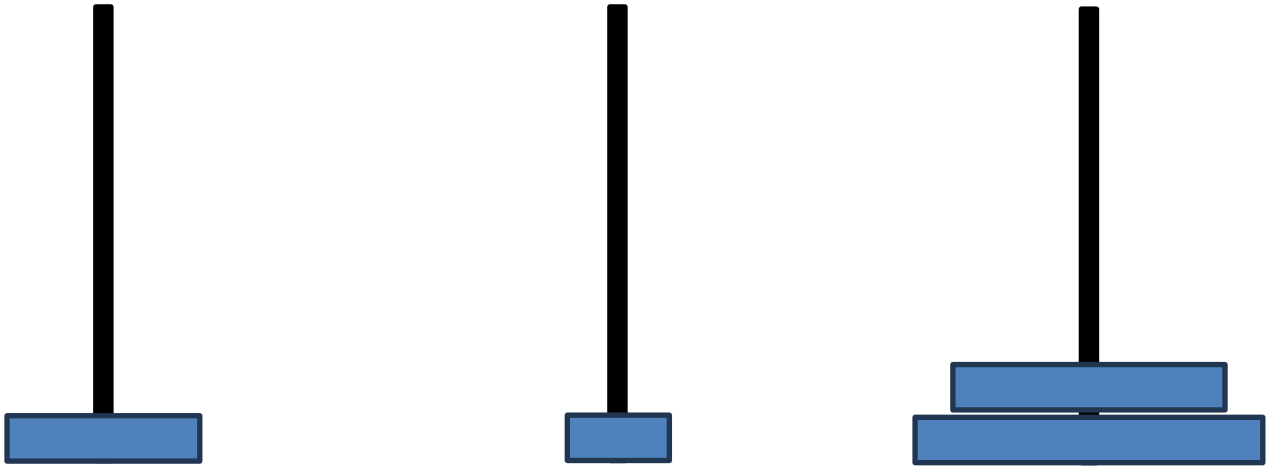
Towers of Hanoi - Example



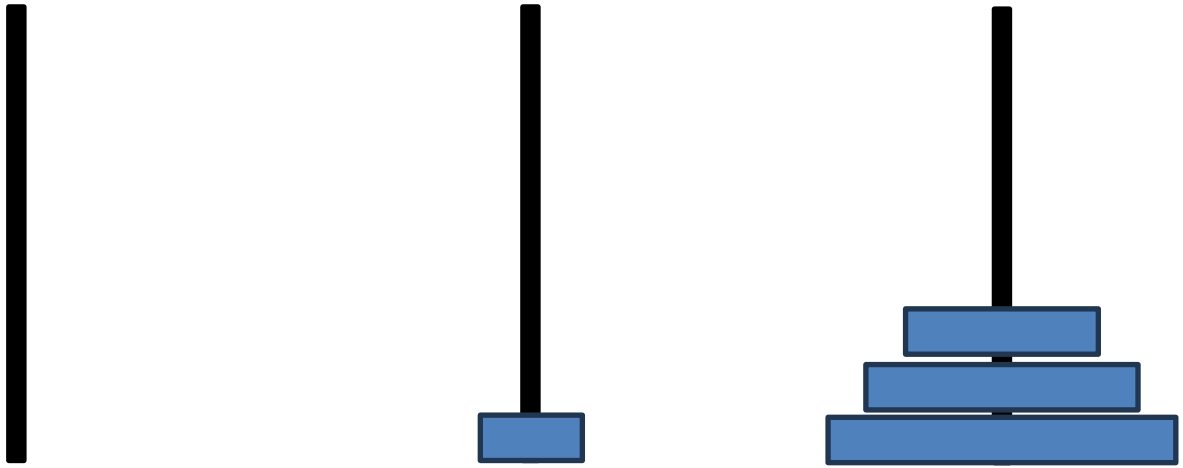
Towers of Hanoi - Example



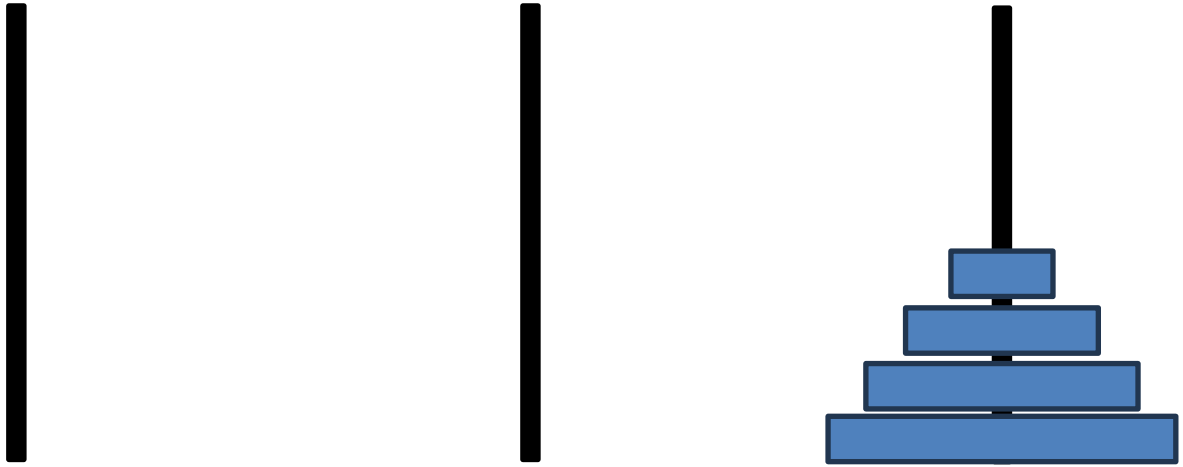
Towers of Hanoi - Example

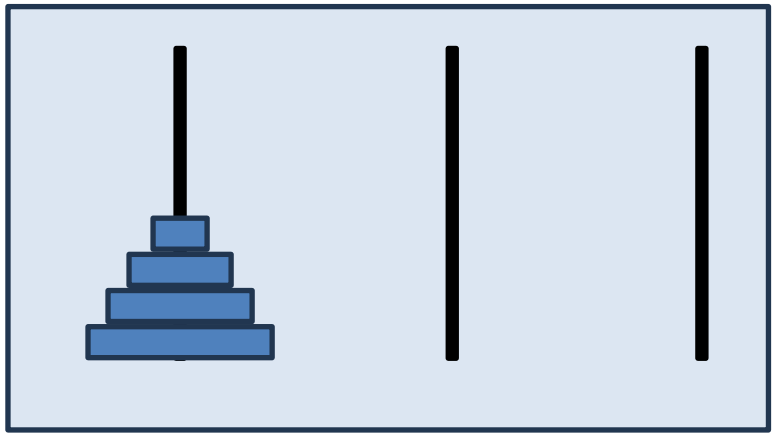


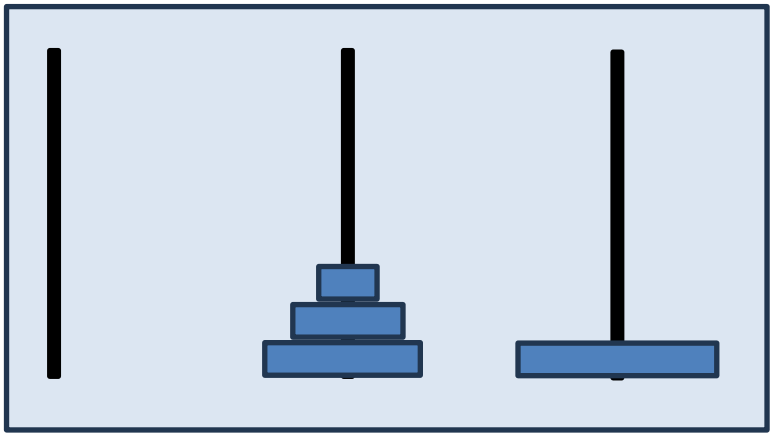
Towers of Hanoi - Example



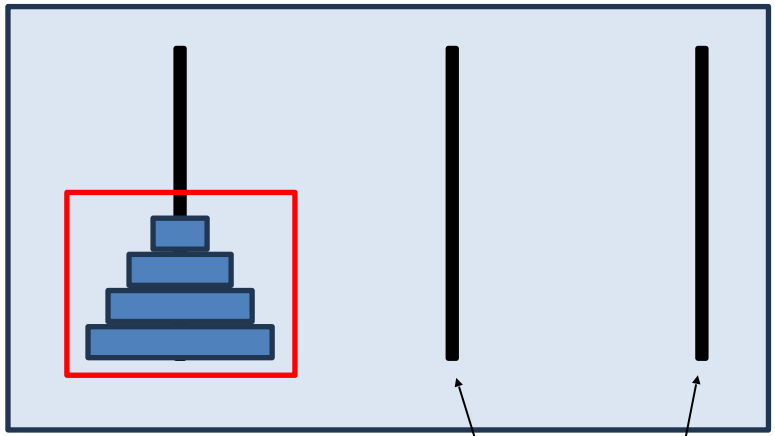
Towers of Hanoi - Example

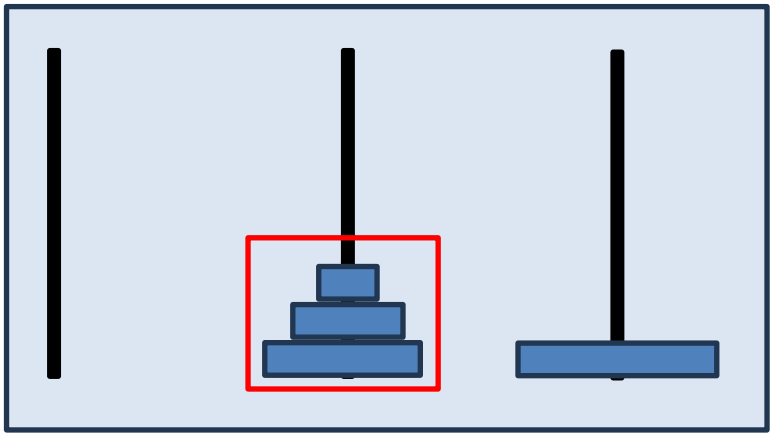




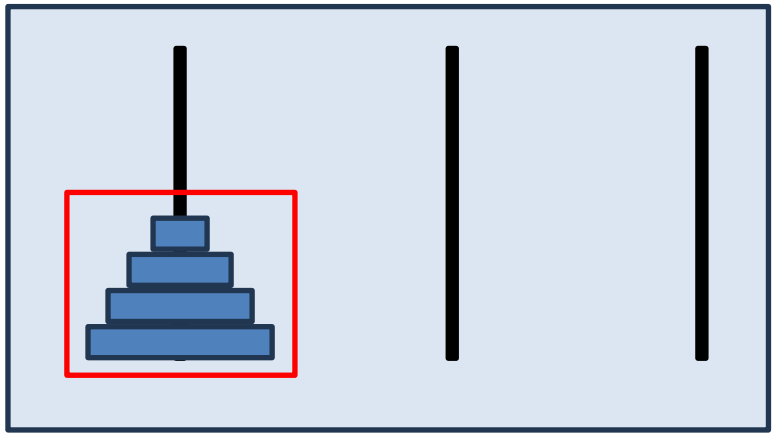


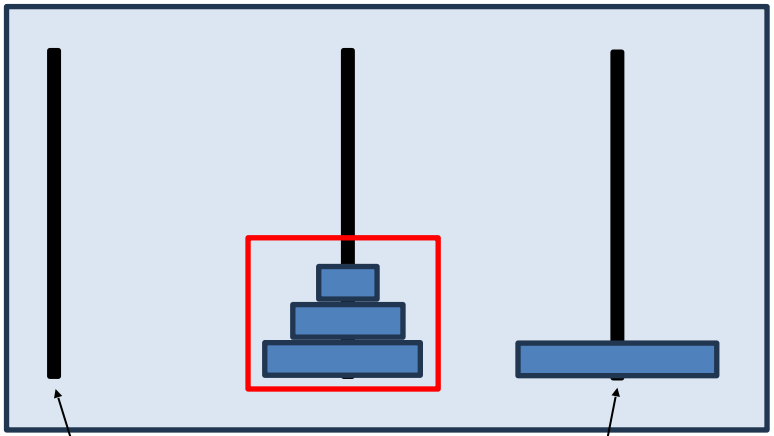
What do the two constellations have in common?



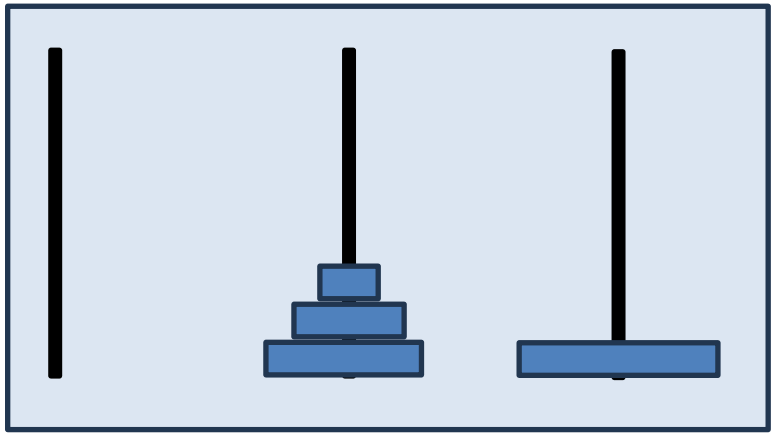


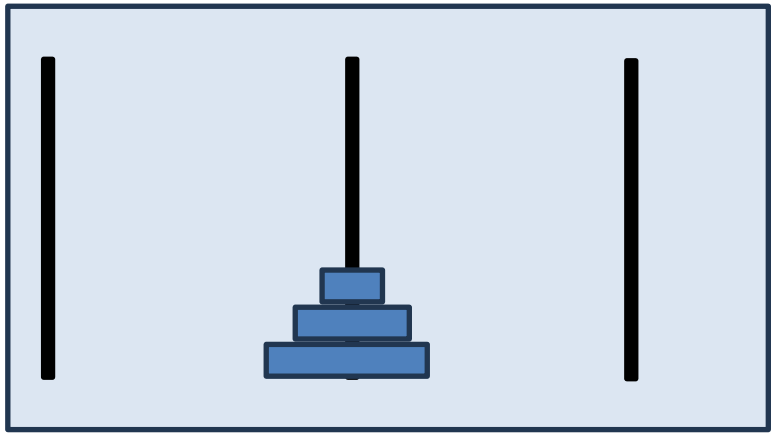
We can place each disk in the red box on the remaining rods.



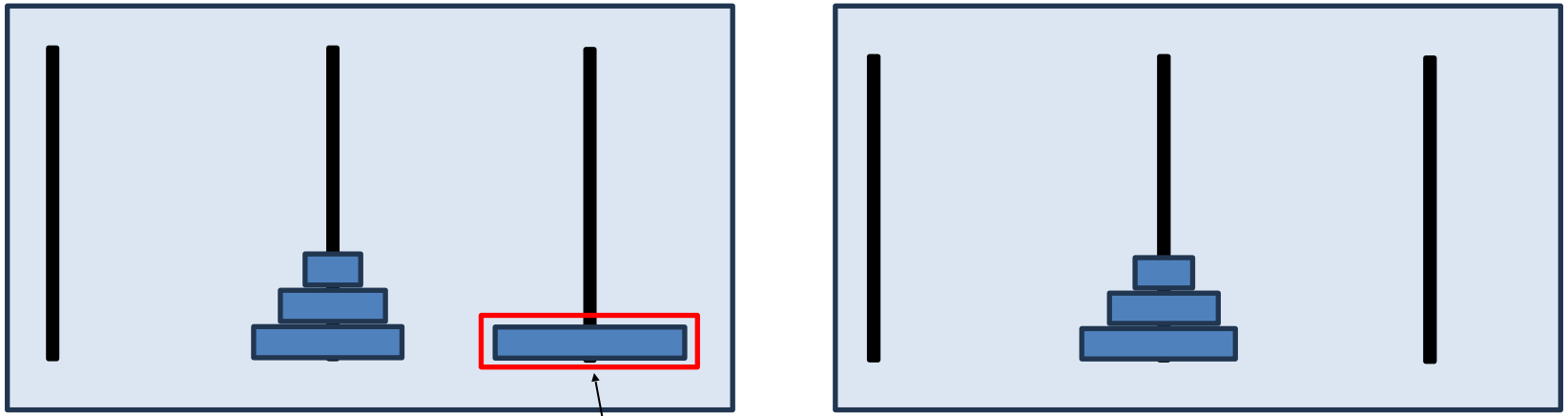


We can place each disk in the red box on the remaining rods.



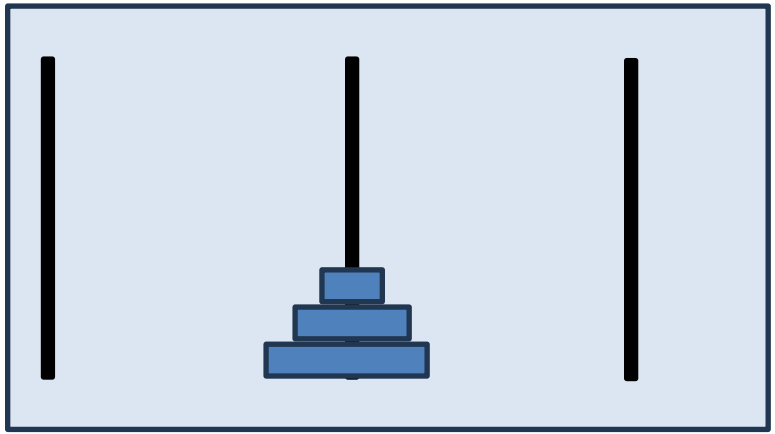


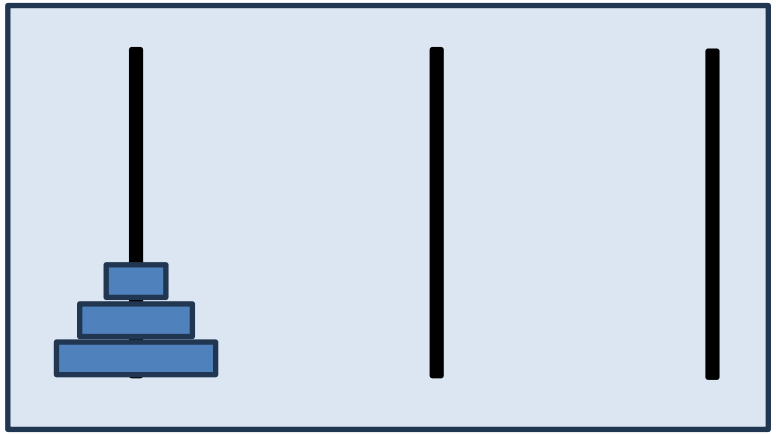
Can the constellations be solved in the same way?



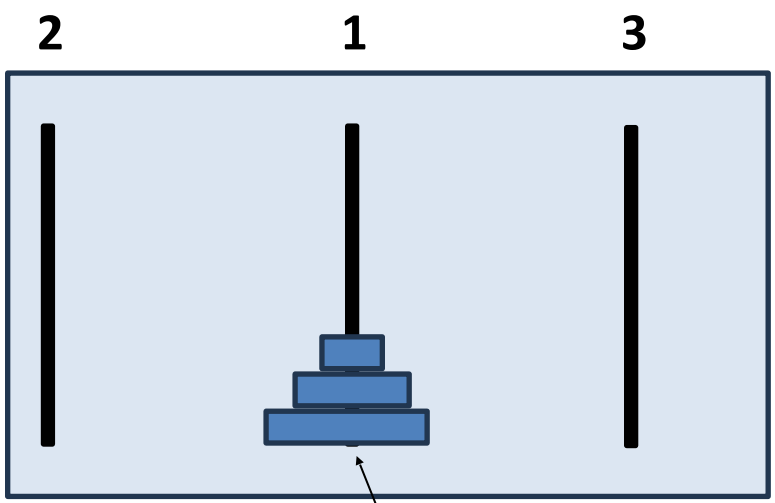
This disk does not change the problem. It is the largest disk, so we can ignore it and only consider the remaining disks.

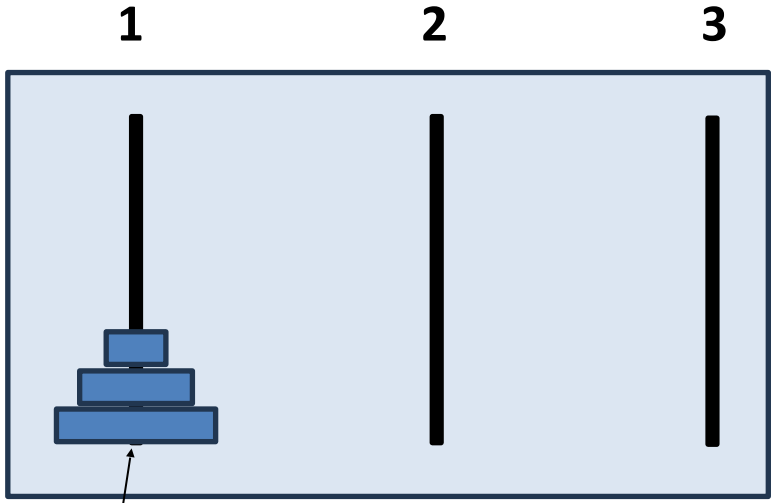
Reduction



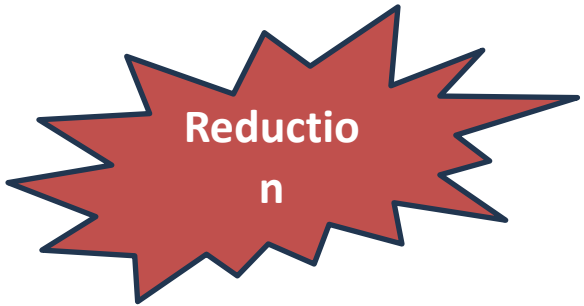


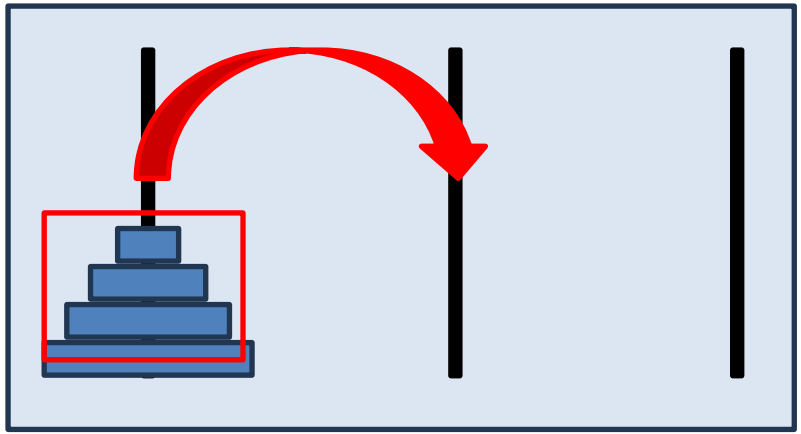
Can the constellations be solved in the same way?

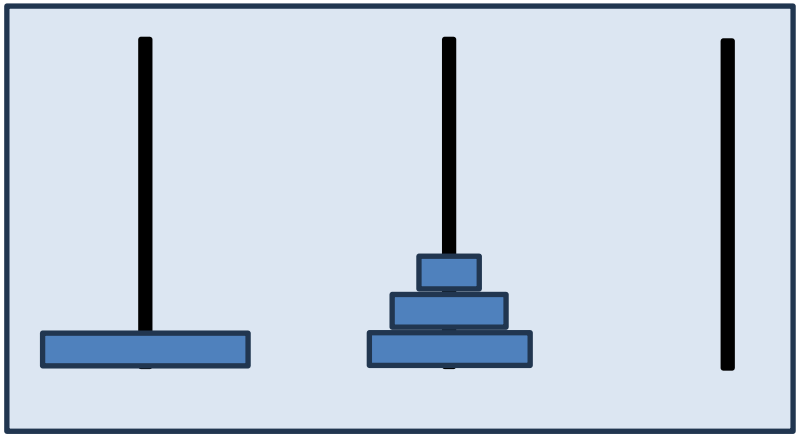




This is the same problem if we number the rods differently.



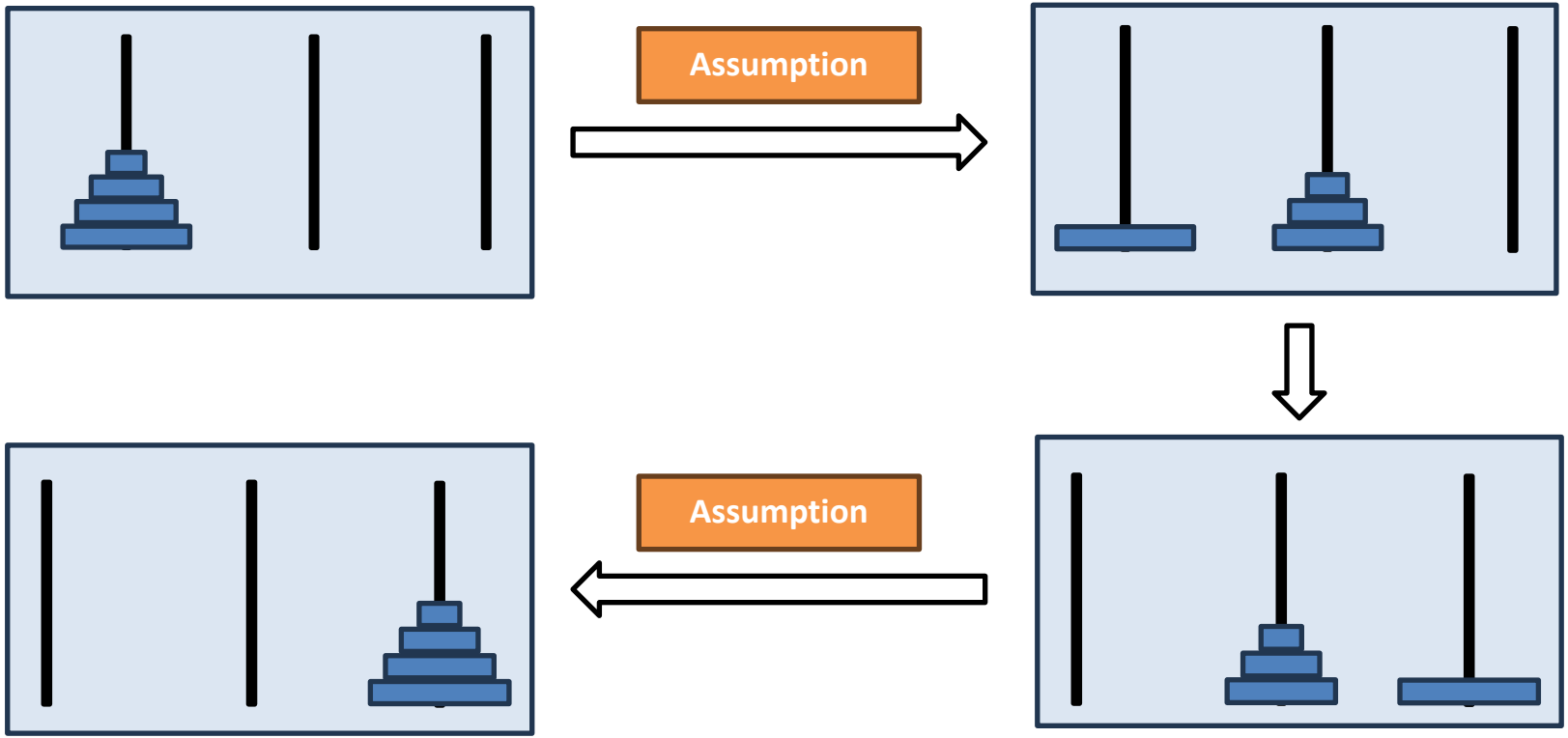




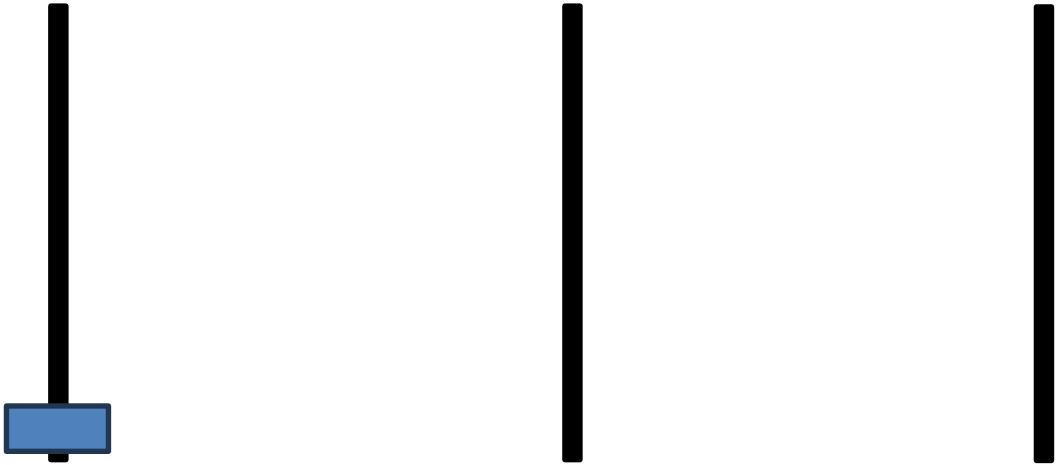
We now assume that we already know how to move a stack of size three onto a "free" rod, if we have two "free" rods.

Assumption

We can solve the problem now!

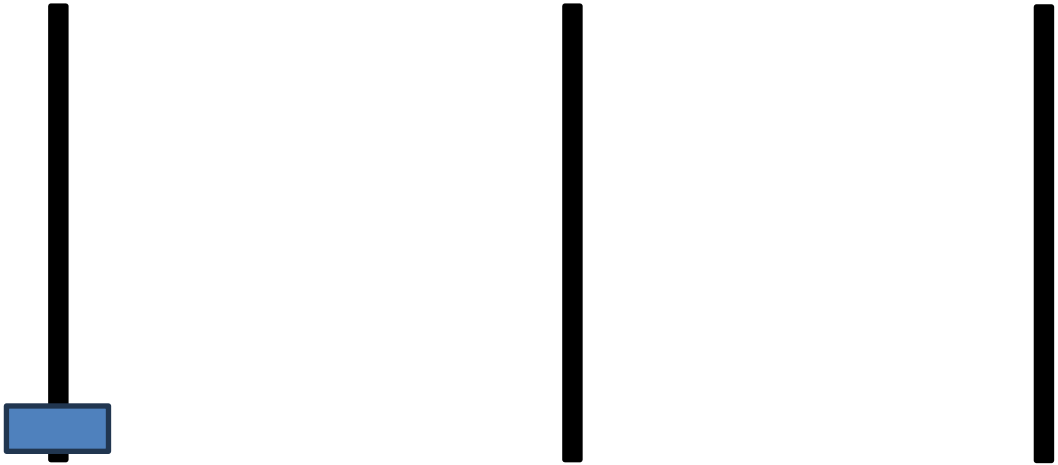


Is the assumption correct?



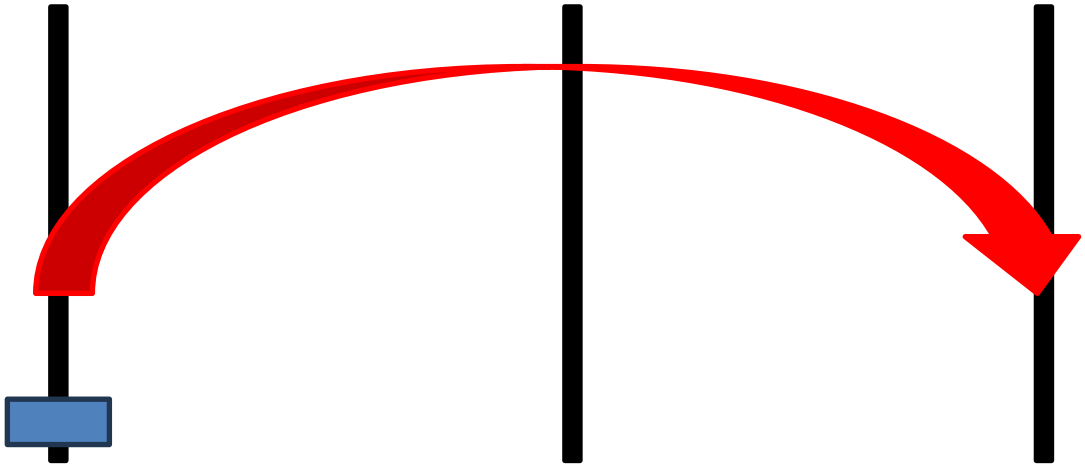
Only in one case!

Is the assumption correct?

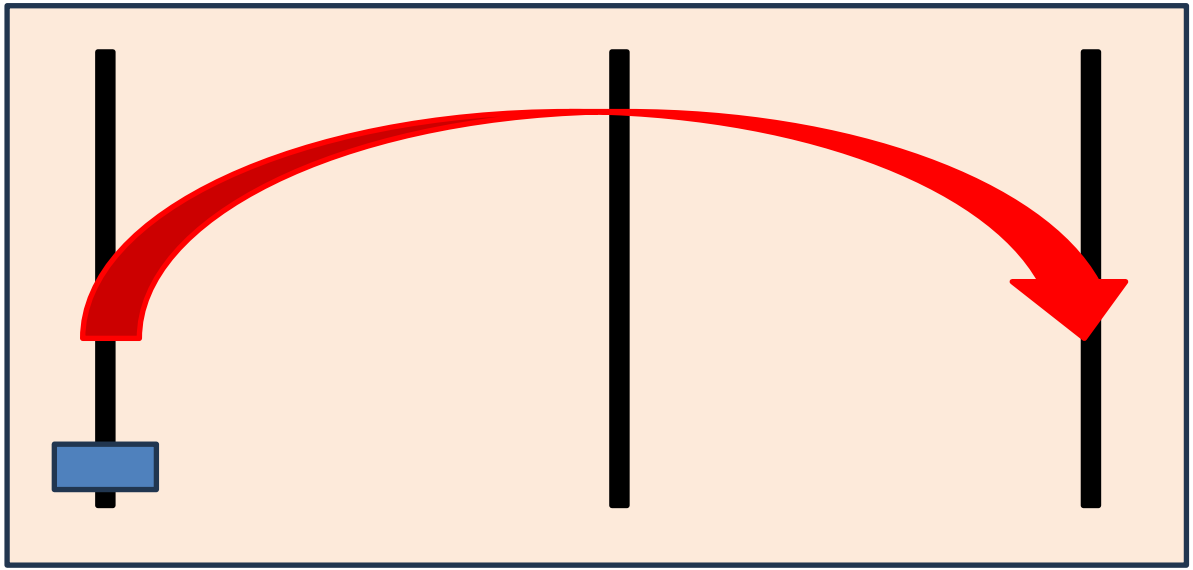


But that is enough!

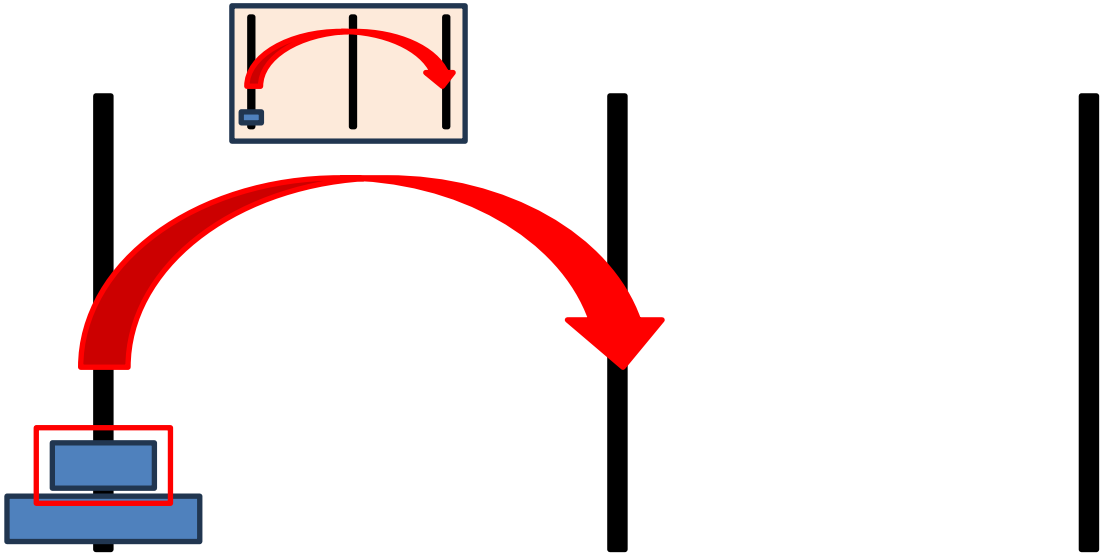
Towers of Hanoi – Step by Step



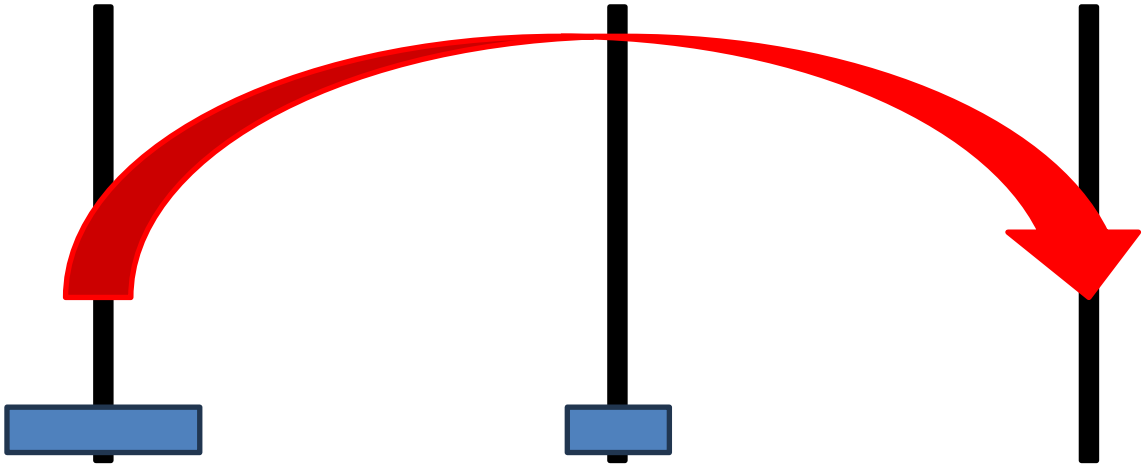
Towers of Hanoi – Step by Step



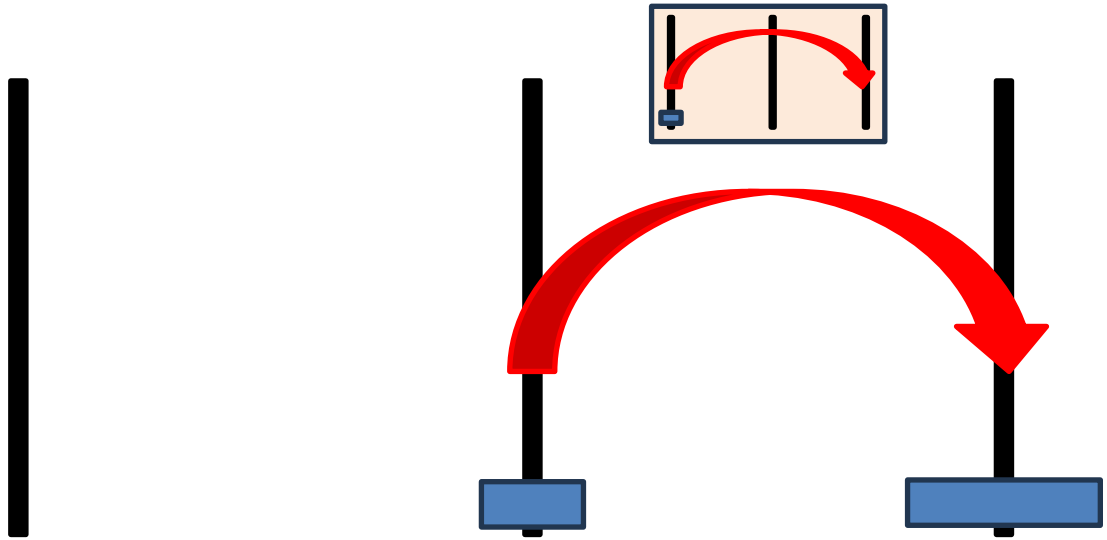
Towers of Hanoi – Step by Step



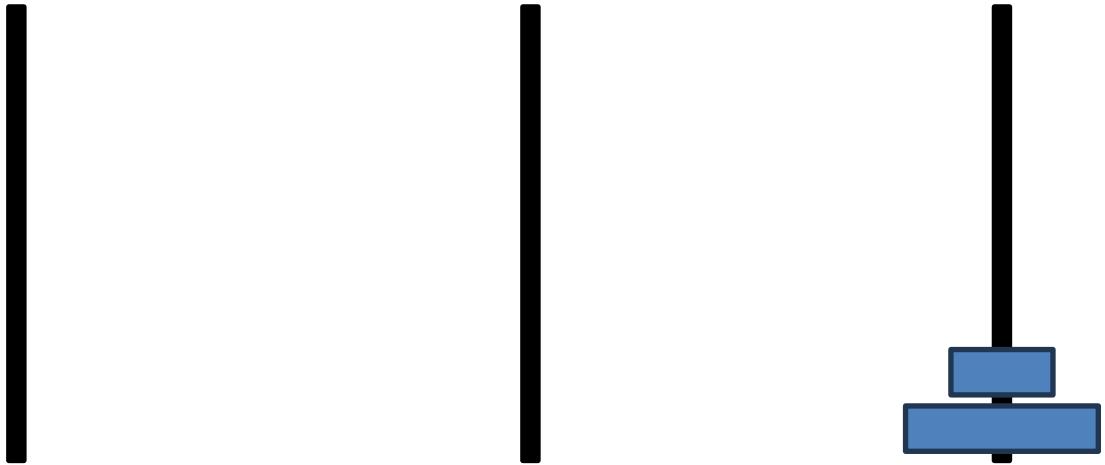
Towers of Hanoi – Step by Step



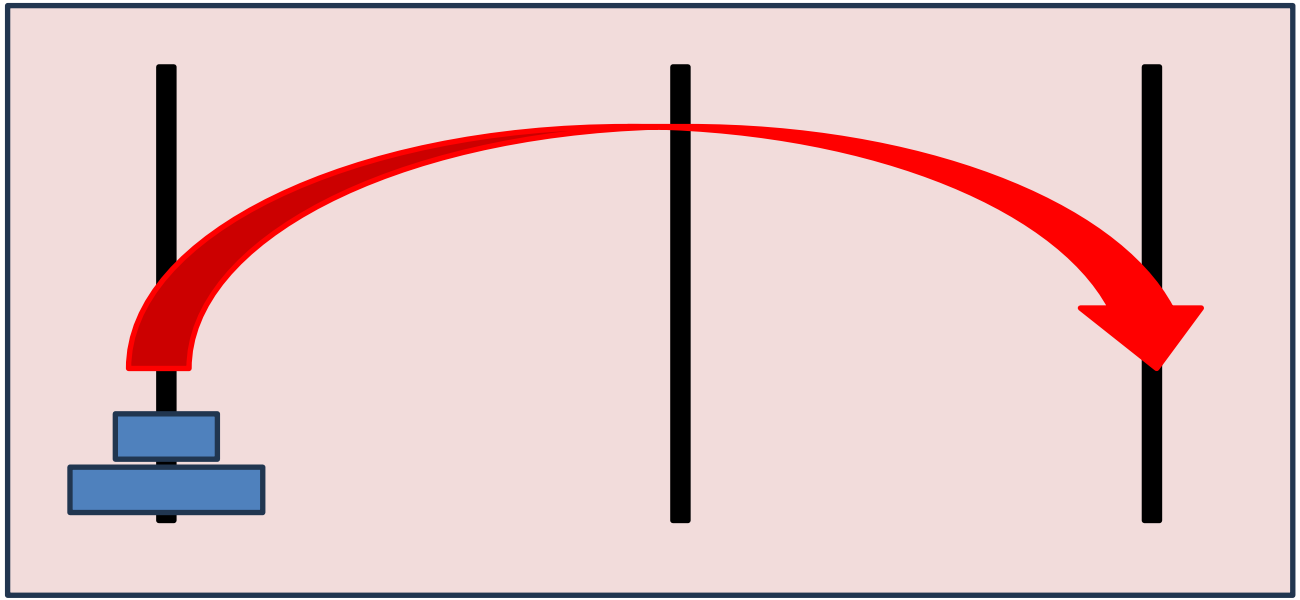
Towers of Hanoi – Step by Step



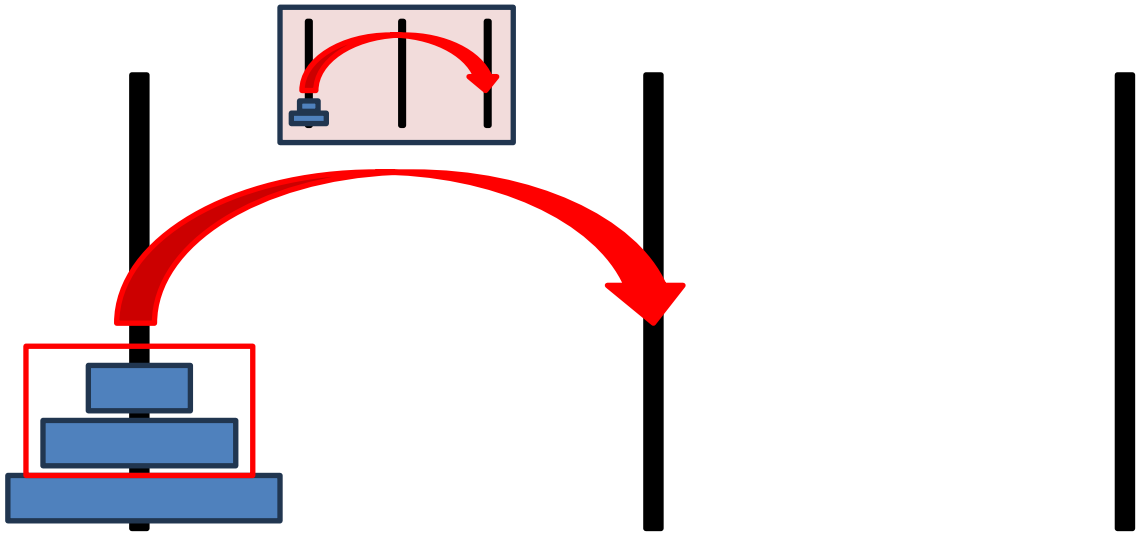
Towers of Hanoi – Step by Step



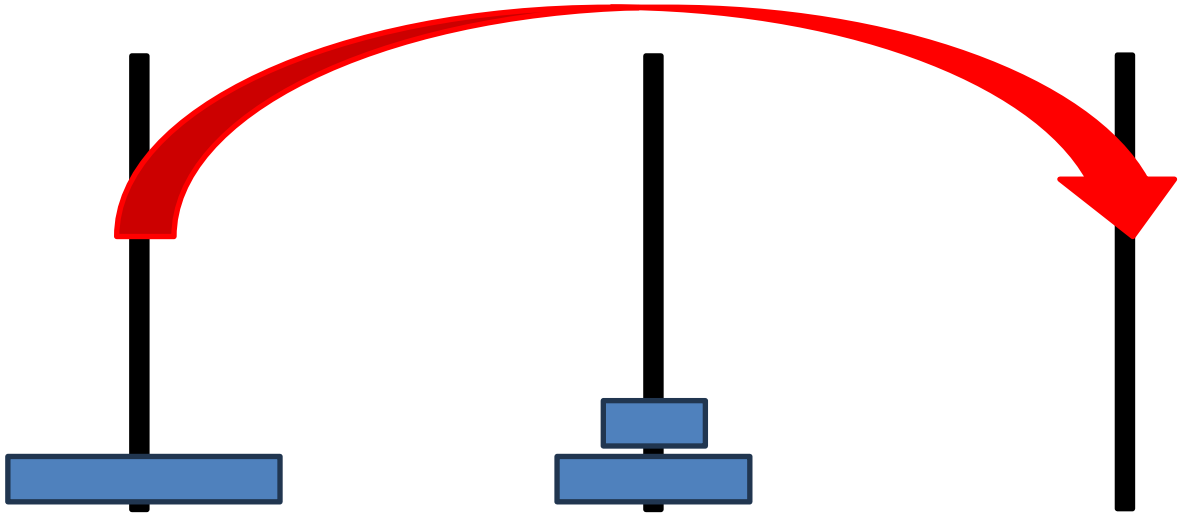
Towers of Hanoi – Step by Step



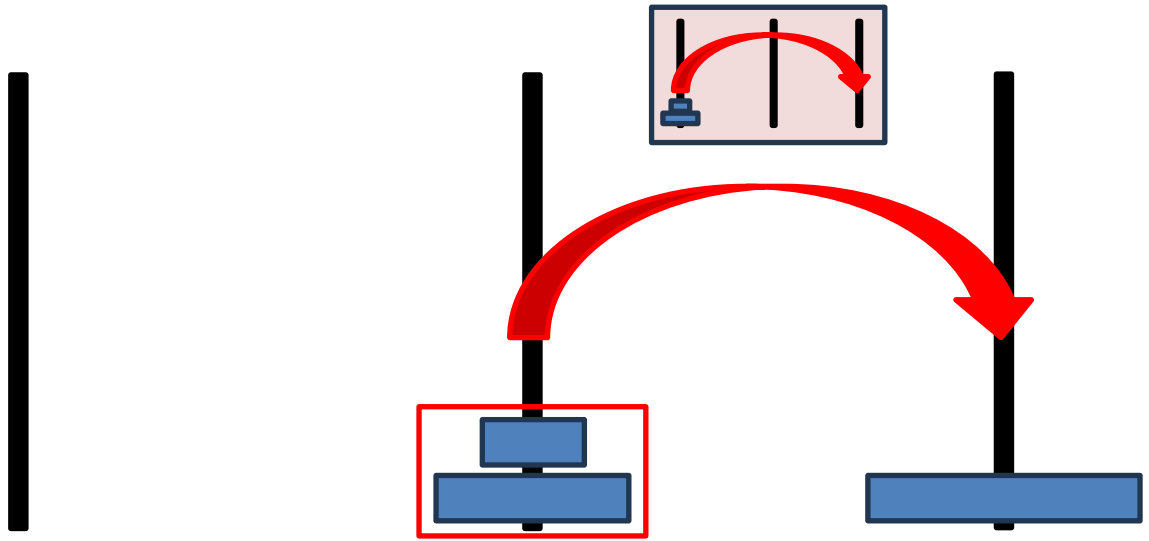
Towers of Hanoi – Step by Step



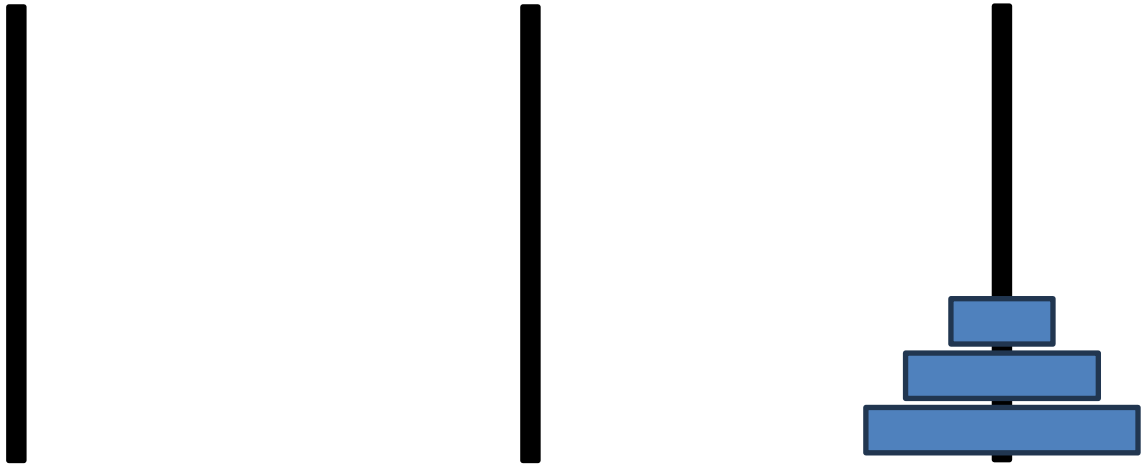
Towers of Hanoi – Step by Step



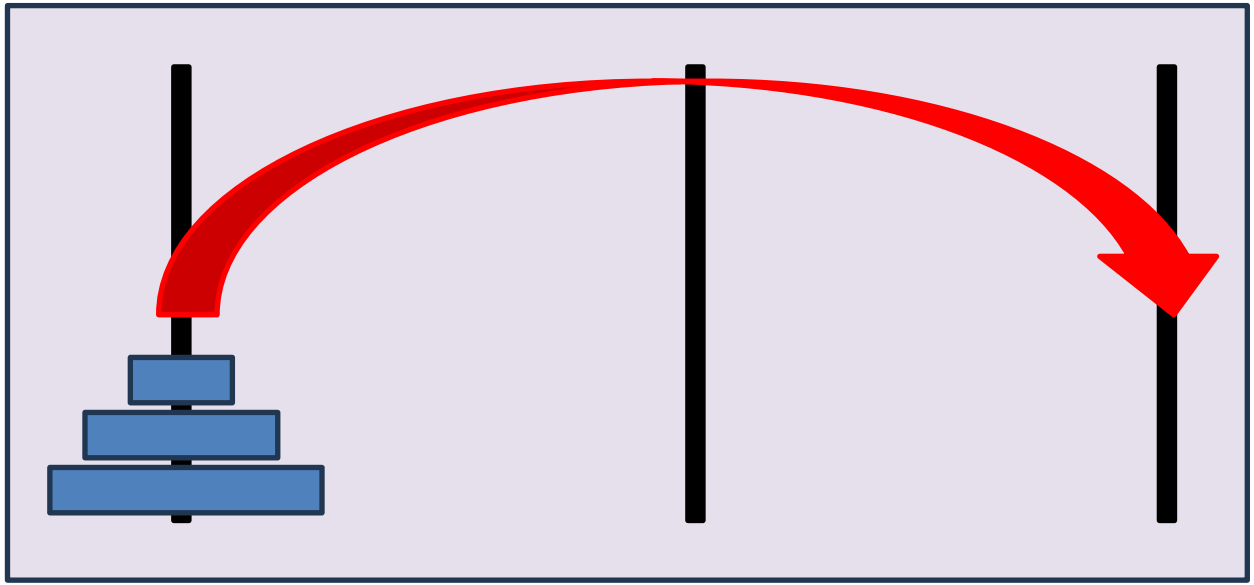
Towers of Hanoi – Step by Step



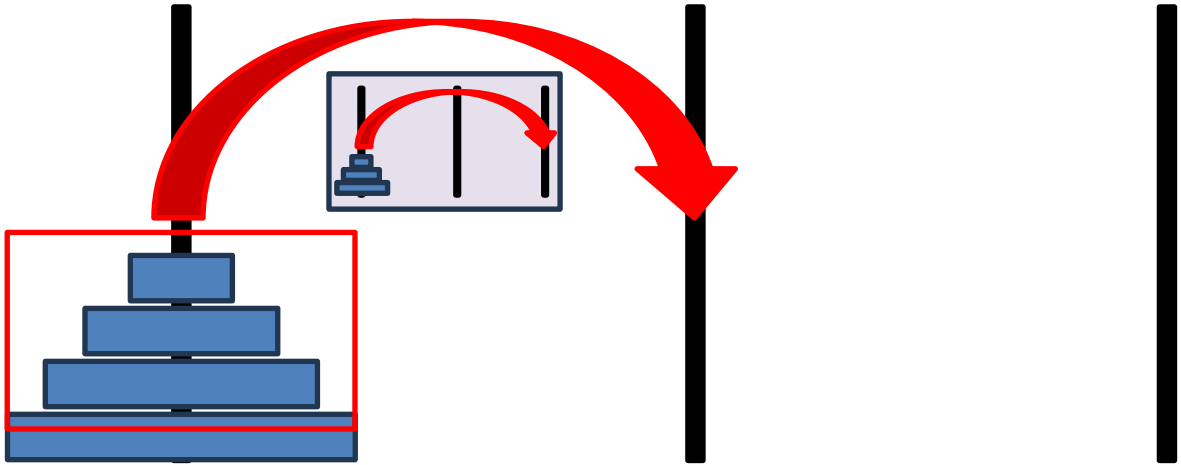
Towers of Hanoi – Step by Step



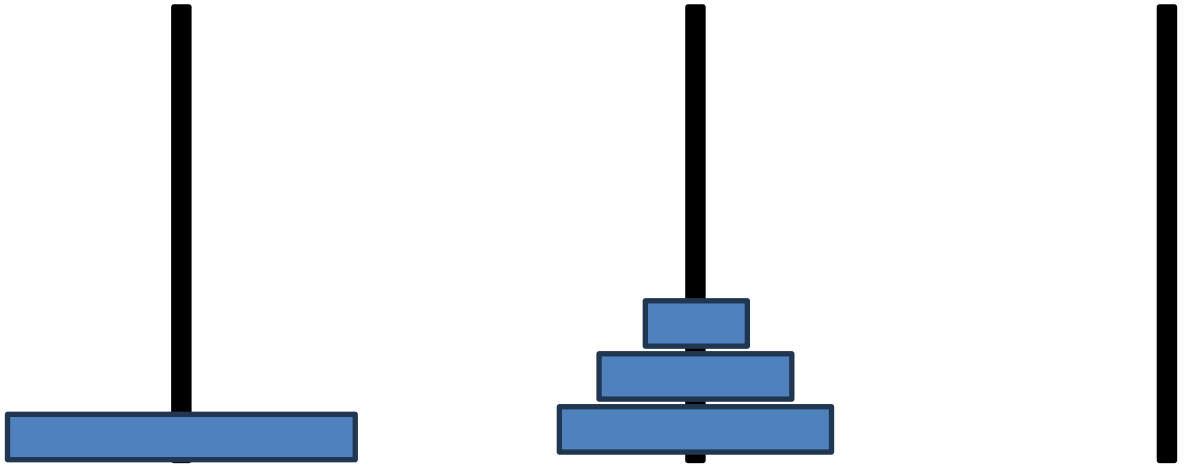
Towers of Hanoi – Step by Step



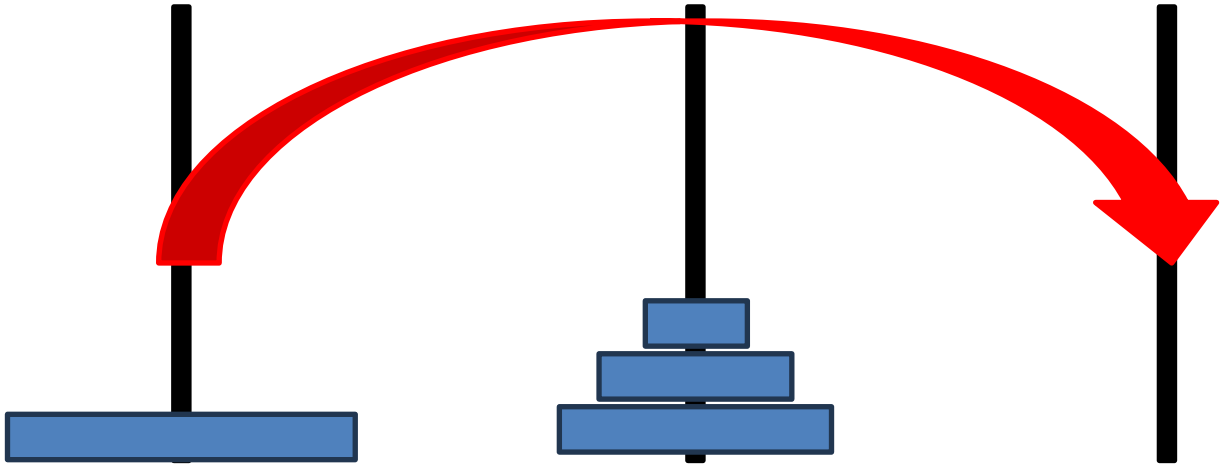
Towers of Hanoi – Step by Step



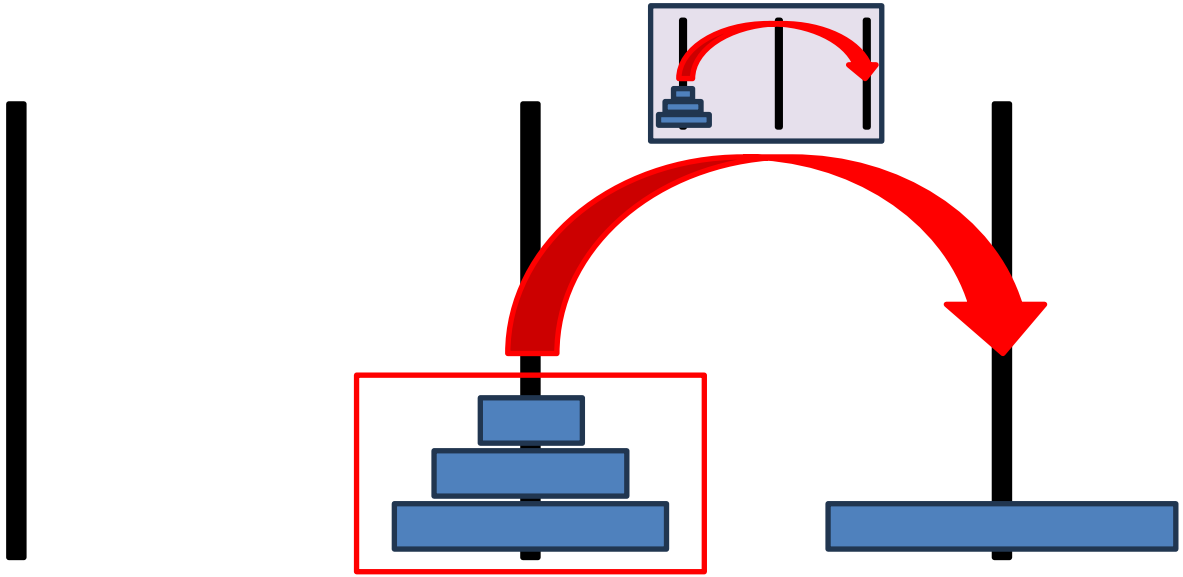
Towers of Hanoi – Step by Step



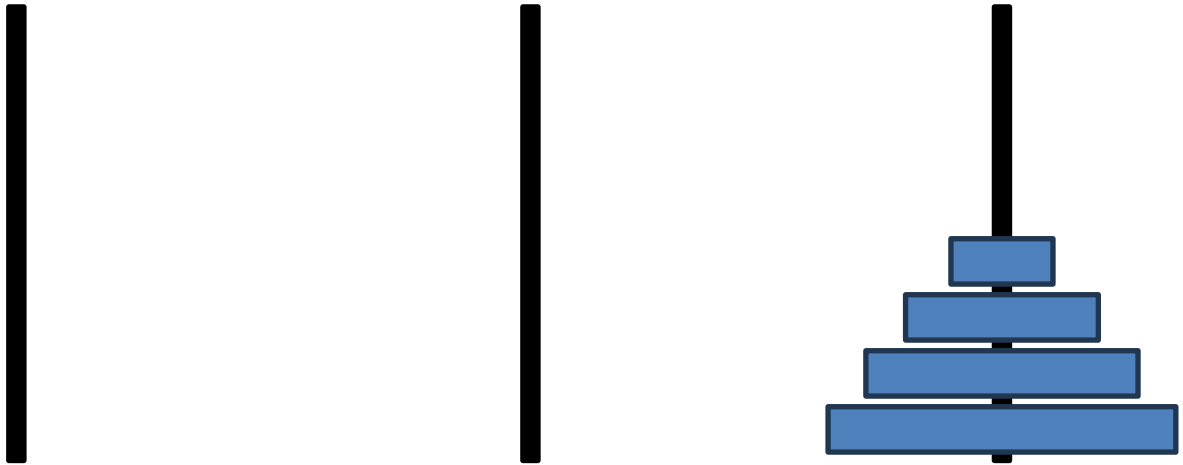
Towers of Hanoi – Step by Step



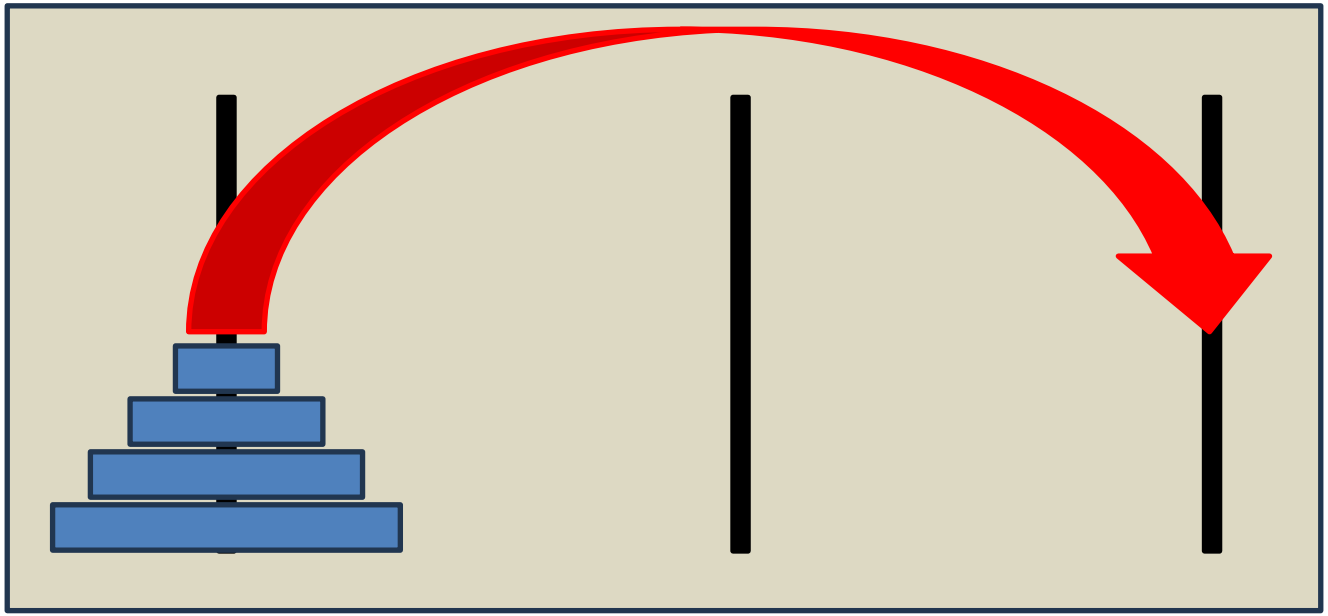
Towers of Hanoi – Step by Step



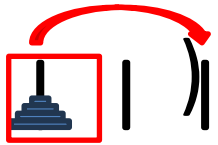
Towers of Hanoi – Step by Step



Towers of Hanoi – Step by Step



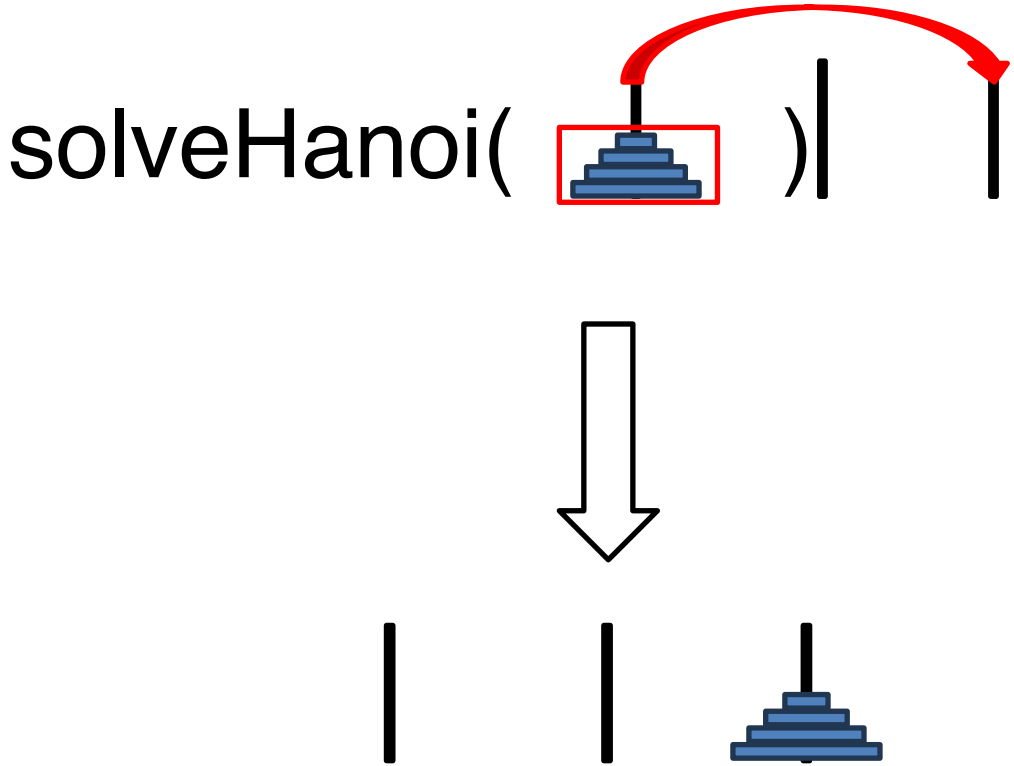
Towers of Hanoi – Methods

What should solveHanoi() be able to do?

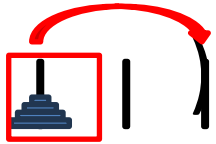
Input: Hanoi constellation with two "free" rods and a (partial) stack that should be moved onto a "free" rod.

- A rod is free if the stack to be moved contains only smaller disks than the disks on that rod.

Towers of Hanoi – Methods



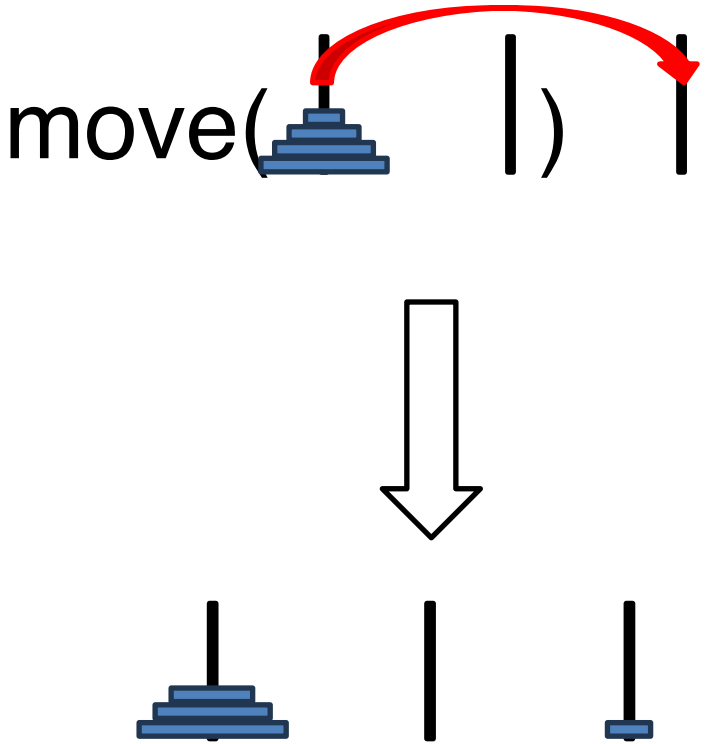
Towers of Hanoi – Methods

What should solveHanoi() be able to do?

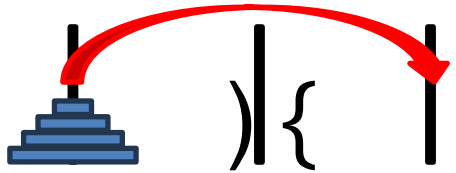
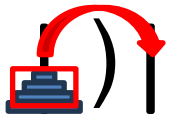
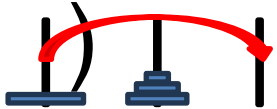
Input: Hanoi constellation with at least one "free" rod and one disk that should be moved onto a "free" rod.

- A rod is free if the stack to be moved contains only smaller disks than the disks on that rod.

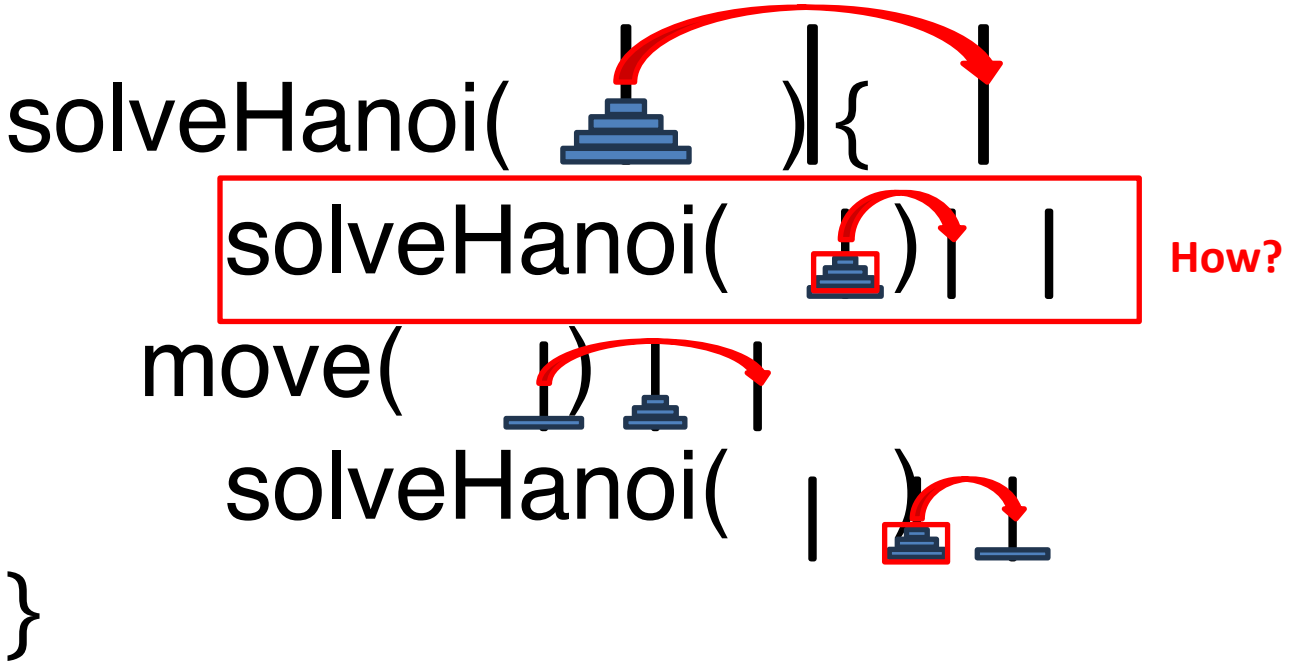
Towers of Hanoi – Methods



Towers of Hanoi – Program

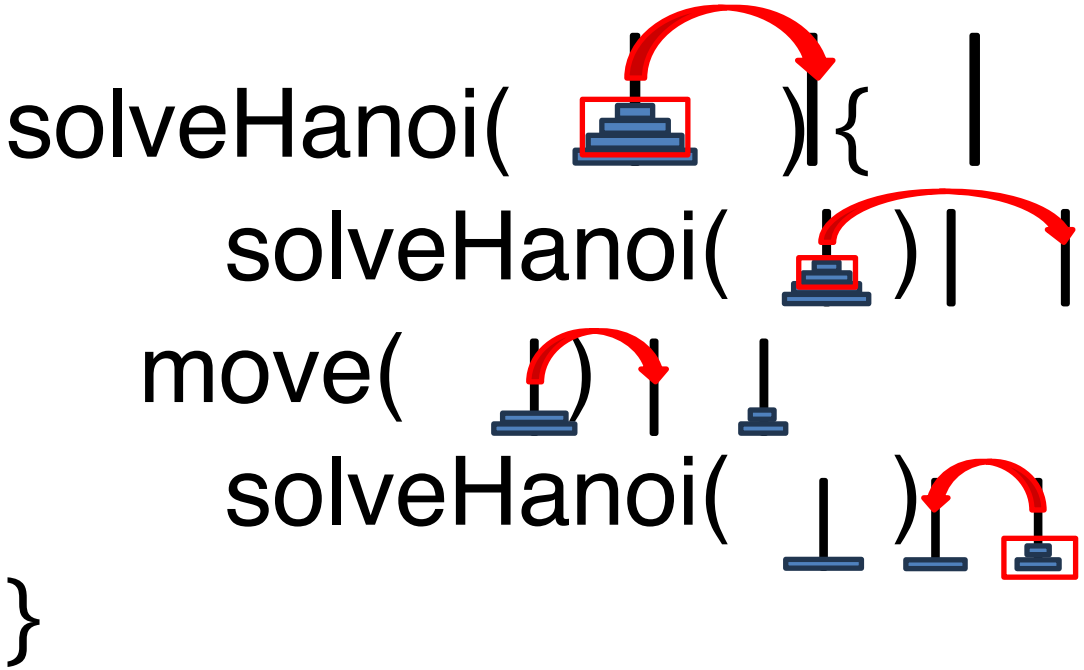
```
solveHanoi(    ){
    solveHanoi(    )
    move(    )
    solveHanoi(    )
}
```

Towers of Hanoi – Program

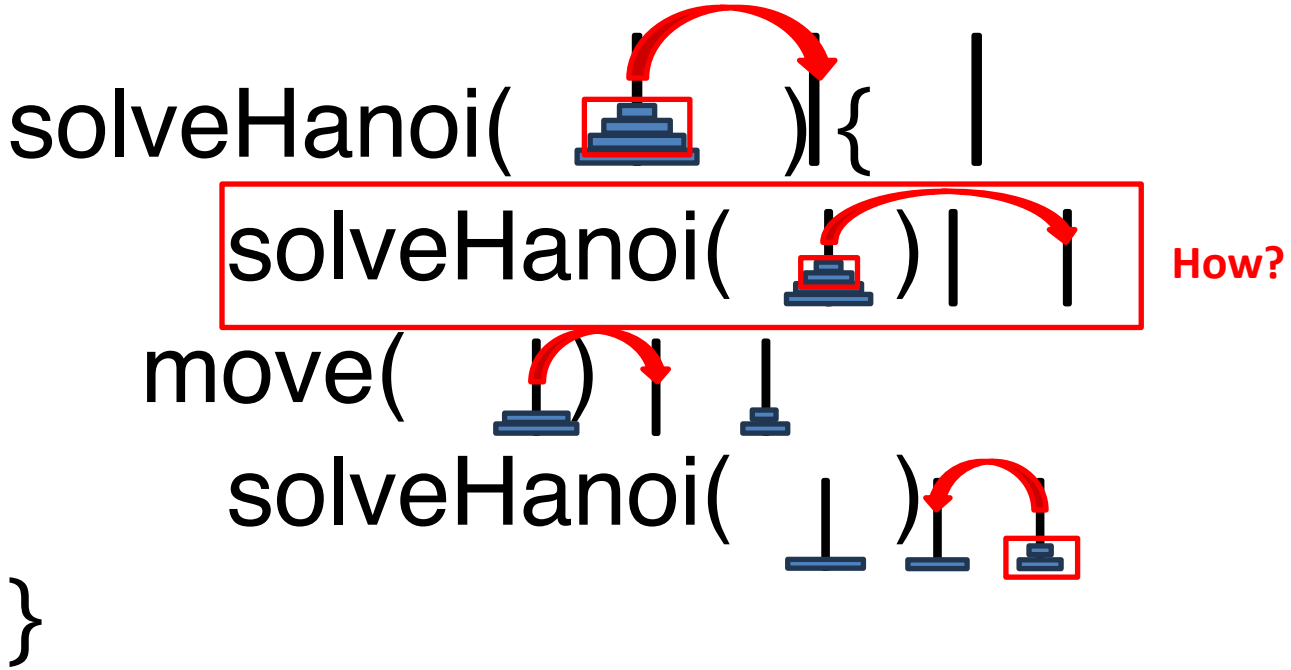


How?

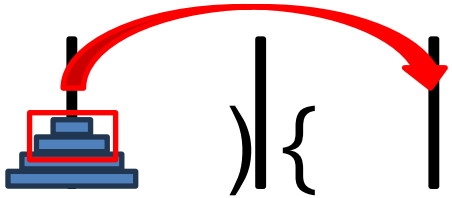
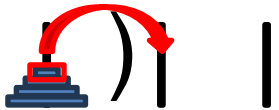
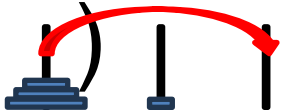
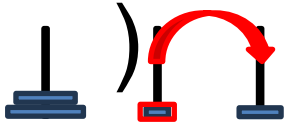
Towers of Hanoi – Program



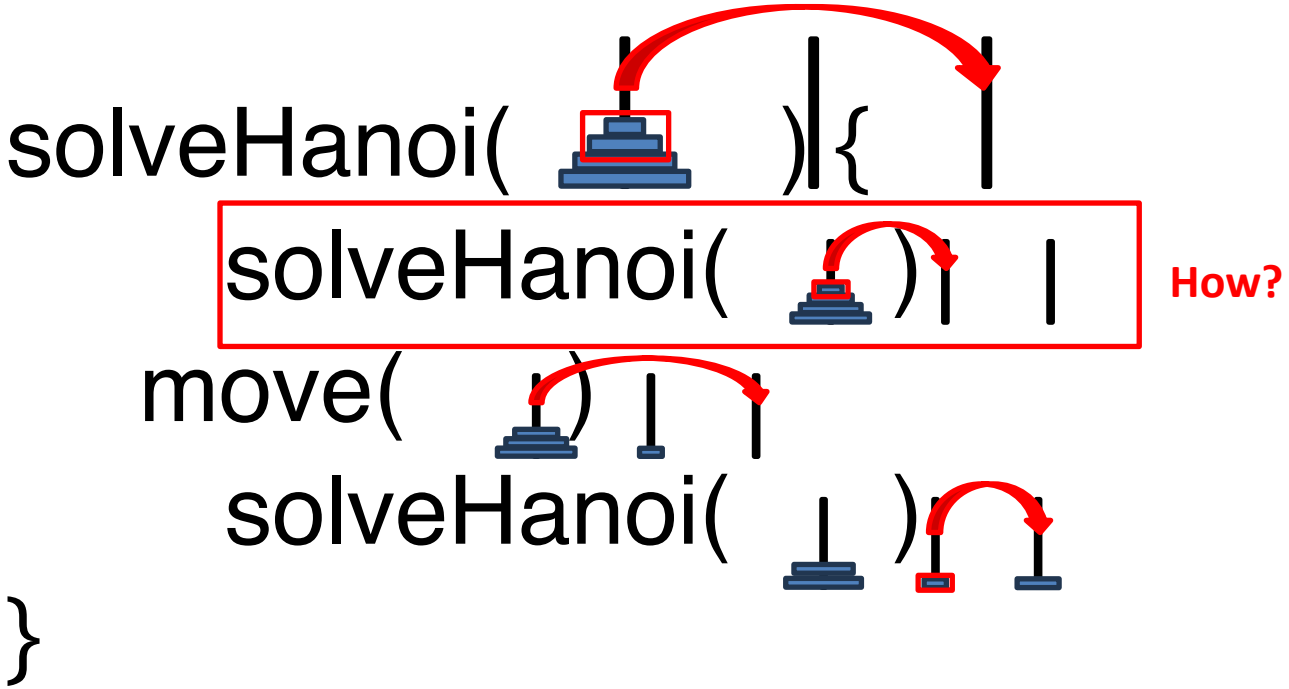
Towers of Hanoi – Program



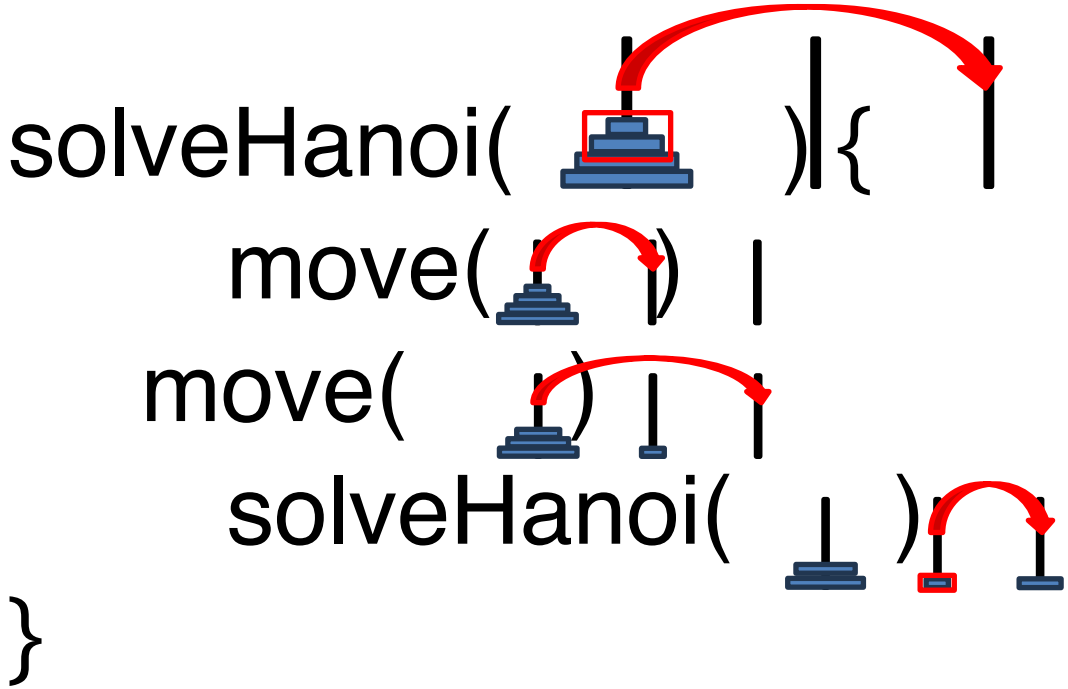
Towers of Hanoi – Program

```
solveHanoi(    ){
    solveHanoi(    )
    move(    )
    solveHanoi(    )
}
```

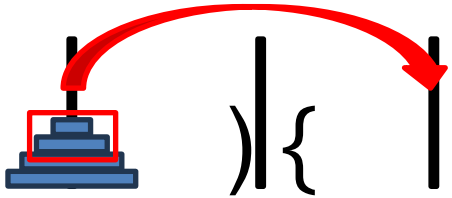
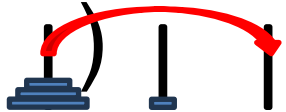
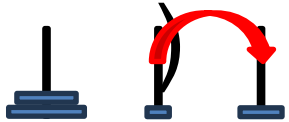
Towers of Hanoi – Program



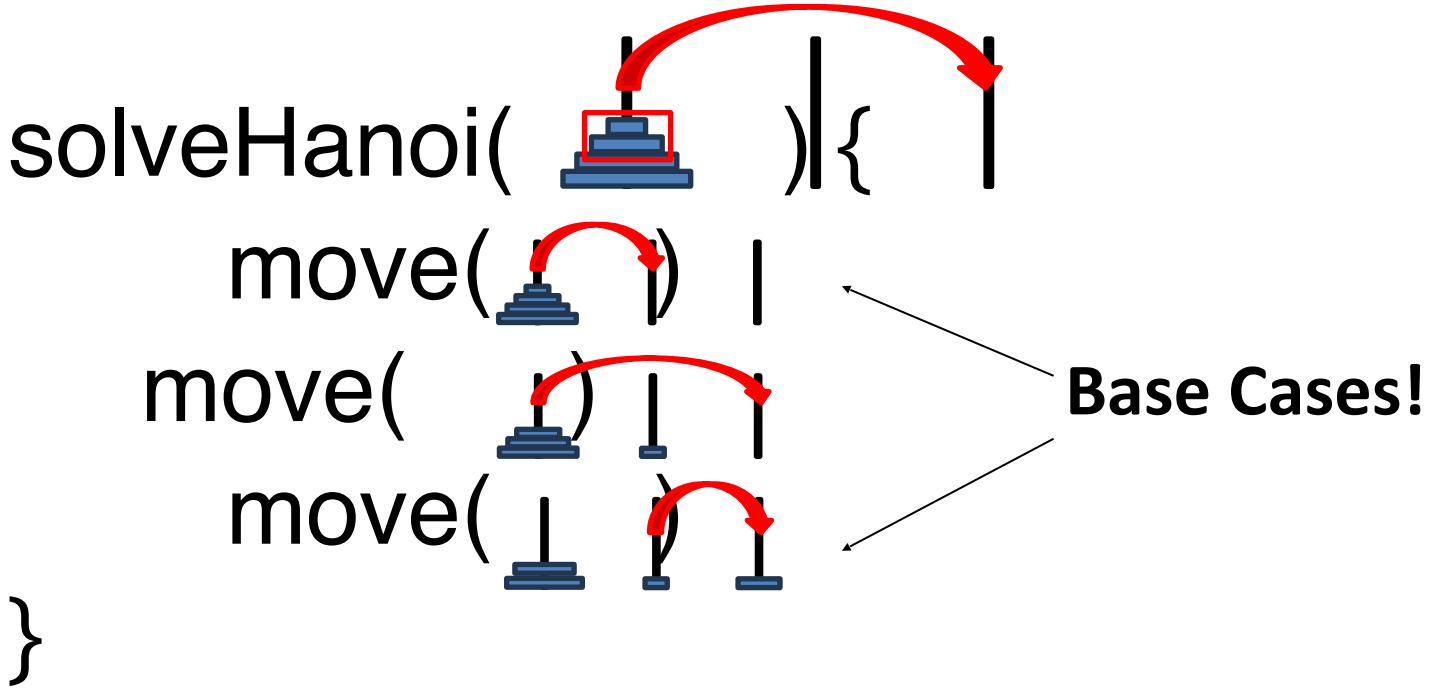
Towers of Hanoi – Program



Towers of Hanoi – Program

```
solveHanoi(    ){
    move(    )
    move(    )
    move(    )
}
```

Towers of Hanoi – Program



Towers of Hanoi – As a Program

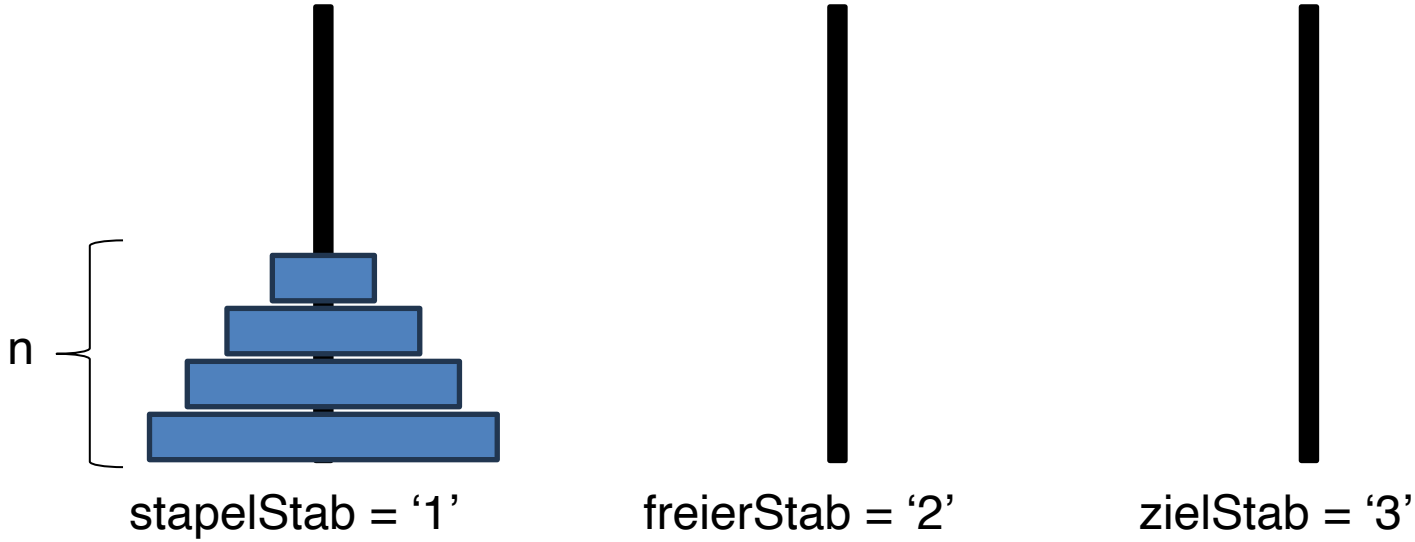
We understand how the problem can be solved.

The how is usually the most important part.

First: understand the problem.

Then: implement the program.

Towers of Hanoi – As a Program



Towers of Hanoi – As a Program

Method 1: solveHanoi(int n, char stapelStab, char freierStab, char zielStab)

Method 2: move(int n, char stapelStab, char freierStab, char zielStab)

Towers of Hanoi – As a Program

```java
public class TowerOfHanoi {
    public static void move(int n, char stapelStab, char freierStab, char zielStab) {
        System.out.println("Scheibe " + n + " wurde von Stab " + stapelStab + " zu Stab " + zielStab + " bewegt.");
    }
    public static void solveHanoi(int n, char stapelStab, char freierStab, char zielStab) {
        if(n == 1) { // Base Case
            move(n, stapelStab, freierStab, zielStab);
        } else { // Step Case
            solveHanoi(n - 1, stapelStab, zielStab, freierStab);
            move(n, stapelStab, freierStab, zielStab);
            solveHanoi(n - 1, freierStab, stapelStab, zielStab);
        }
    }
    public static void main(String[] args) {
        int scheiben = 4;

        solveHanoi(scheiben, '1', '2', '3');
    }
}
```

Towers of Hanoi – As a Program

Link to the code: https://
lec.inf.ethz.ch/infk/eprog/
2026/exercises/additionals/
TowerOfHanoi.java

